The present work was submitted to the

Chair of Intelligence in Quality Sensing | WZL | RWTH Aachen University.

Diese Arbeit wurde vorgelegt am Lehrstuhl für Informations-, Qualitäts- und Sensorsysteme in der Produktion | WZL | RWTH Aachen.

Integration and Validation of Whole-Body Model Predictive Control for a Mobile Manipulator in Line-less Assembly

Integration und Validierung einer ganzkörperbasierten modellprädiktiven Regelung für einen mobilen Manipulator in der linienlosen Montage

Project Thesis / **Projektarbeit**

Presented by / **Vorgelegt von**

Xiangming You

Matriculation No. 440593

Supervised by / **Betreut von** Guillaume Tréheux, M.Sc.

1st Examiner / **1. Prüfer** Univ.-Prof. Dr.-Ing. Robert H. Schmitt

Aachen, Tuesday 13th October, 2026

Inhalt und Ergebnis dieser Arbeit sind ausschließlich zum internen Gebrauch bestimmt. Alle Urheberrechte liegen bei der RWTH Aachen. Ohne ausdrückliche Genehmigung des betreuenden Lehrstuhls ist es nicht gestattet, diese Arbeit oder Teile daraus an Dritte weiterzugeben.

The content and results of this thesis are intended exclusively for internal use. All copyrights are held by RWTH Aachen University. Without the express permission of the supervising department, it is not permitted to pass this thesis or parts thereof to third parties.

I Contents

I Contents i

II List of Figures ii

III List of Tables 1

1 Introduction to Line-less Mobile Assembly Systems 2

1.1 Line-less Mobile Assembly Systems and Their Implications for Motion Planning . . 2

1.2 The Need for Whole-Body Motion Planning: From Sequential to Unified Approaches 5

1.3 Objectives and Structure of This Work 7

2 State of the Art in Whole-Body Motion Planning for Mobile Manipulators 10

2.1 Classification of Motion Planning Approaches 10

2.2 Overview of Holistic Motion Planners 12

2.3 Comparison of Motion Planning Algorithms and Selection of a Suitable Algorithm . 16

3 Integration of OCS2 into the HERo Mobile Manipulator 18

3.1 HERo Platform: Differential-Drive Base and UR10e Manipulator 18

3.2 Distributed System Architecture for Real-Time MPC 22

3.3 Whole-Body Model and Parameterisation 24

3.4 Hardware Interfaces and ROS Control Layer Integration 32

4 Simulation-Based Validation 37

4.1 Validation Methodology and Evaluation Metrics 37

4.2 Kinematic Closed-Loop Simulation (Dummy MRT) 41

5 Hardware Deployment and Preliminary Experimental Results 50

5.1 Deployment on the HERo Platform 50

5.2 Deployment Challenges and Mitigations 51

5.3 Preliminary End-Effector Tracking Results 55

6 Conclusion and Outlook 59

6.1 Summary 59

6.2 Critical Reflection 61

6.3 Recommendations for Resolving Present Issues 62

References 65

II List of Figures

1.1 Station-bound assembly line and line-less mobile assembly system 3
1.2 Mobile manipulator: base and arm degrees of freedom 4
1.3 Decoupled versus whole-body command of a mobile manipulator 7

2.1 Classification axes for mobile-manipulator motion planning 11
2.2 Receding-horizon principle of MPC 13
2.3 Structure of the OCS2 framework 14

3.1 The HERo platform: UR10e arm on the *freeMoVe* base with its red drive unit. The teach pendant lies on the top plate in (a) and (b). The operator panel next to the shoulder in (c) is the `hmi` link of the kinematic description; the UR control box sits inside the chassis below it. 19
3.2 Kinematic tree and frame convention of HERo 20
3.3 Complete transform tree of HERo 21
3.4 Deployment of OCS2 on HERo 23
3.5 Structure of the OCS2 objective on HERo 26
3.6 Relaxed barrier penalty at the configured relaxation width 26
3.7 Joint-limit page of the UR10e safety configuration on the teach pendant, installation `kalimoro`. The Normal-mode ranges equal the bounds in `task_hardware.info`; the Reduced-mode ranges are not used in this work. 27
3.8 Platform bodies and arm attachments that enter the self-collision model. 28
3.9 Arm postures in which the wrist comes close to the chassis 30
3.10 Real-time invariant of the MPC–MRT loop 30
3.11 Reset handshake between the MRT bridge and the MPC node 33
3.12 One cycle of the MRT bridge 35

4.1 Decomposition of the time-aligned end-effector position error 39
4.2 Both directed Hausdorff distances are required 40
4.3 Dummy-MRT closed loop for Experiment A 42
4.4 Eight-corner commanded path for Experiment A 43
4.5 Phase sequence of the waypoint node 45
4.6 RViz view of the dummy loop 48
4.7 Residual of the dummy loop over the measured path 49

5.1 Distribution of last-solve time on the mock and hardware sessions 52
5.2 Policy lag on the hardware lap 53
5.3 Base command inside one policy against the lag at which it is read 54
5.4 Commanded and attained end-effector paths, mock versus hardware 55
5.5 Time-aligned end-effector position error on the measured path 56
5.6 Last-solve time on the mock and hardware sessions 58

6.1 Requirements R1 to R7 against the evidence this thesis provides 61
6.2 Decision settled by the lag-compensation experiment 64

III List of Tables

1.1 Requirements on the motion planner for HERo 8

2.1 Comparison of motion-planning approaches for HERo 16

3.1 Process placement and traffic across the machine boundary 22
3.2 Self-collision pairs before and after the reduction 29
3.3 OCS2 operating point on HERo 31
3.4 Guards implemented in the MRT bridge 36

4.1 Metrics used in the closed-path experiments 41
4.2 Waypoint offsets of the Experiment A box 44
4.3 Residual of the dummy loop on the accepted Experiment A lap 47

5.1 Mock versus hardware scores on the measured Experiment A lap 56
5.2 Per-edge end-effector position error on the measured lap 57

1

Introduction to Line-less Mobile Assembly Systems

1.1 Line-less Mobile Assembly Systems and Their Implications for Motion Planning

Manufacturing operates under shortening product life cycles, growing variant diversity, and demand that fluctuates over the life of a product; a CIRP-level review of product-variety management documents this trend across industries and quantifies the cost it imposes on assembly [ELMA13]. Rigid, station-based assembly lines are dimensioned for a fixed product mix and a stable takt time. When the product or the required volume changes, layout, material flow, and station balancing have to be reconfigured, which is slow and capital-intensive [HÜTT16].

Manufacturing research has answered this tension with a family of paradigms that trade dedicated throughput for adaptability. Flexible Manufacturing Systems provide a fixed set of capabilities that can be re-combined for a pre-defined product family, whereas Reconfigurable Manufacturing Systems are designed so that capacity and functionality themselves can be changed by adding, removing, or rearranging modules [KORE99]. ElMaraghy contrasts the two paradigms and places them on a common axis of changeability, with flexibility bought in advance in the former case and acquired on demand in the latter [ELMA05]. In assembly specifically, this line of work has produced modular station concepts in which workstations are no longer traversed in a fixed sequence [KERN16].

The *Line-less Mobile Assembly System* (LMAS) pushes this development to its limit by removing the fixed station grid altogether [HÜTT19]. Instead of binding products, tools, and workers to a sequence of static stations, an LMAS models all production resources—mobile robots, mobile manipulators, workpiece carriers, and human workers—as *mobile, freely relocatable entities* on a shared shop floor, whose assignment, sequence, and location are planned at run time [BUCK19].

The assembly "line" thus becomes a reconfigurable constellation of resources matched to the current order, and control of that constellation is decentralised so that the system can absorb product changes, demand fluctuations, and disturbances without a physical relayout [BUCK22]. Figure 1.1 contrasts this arrangement with a station-bound line.

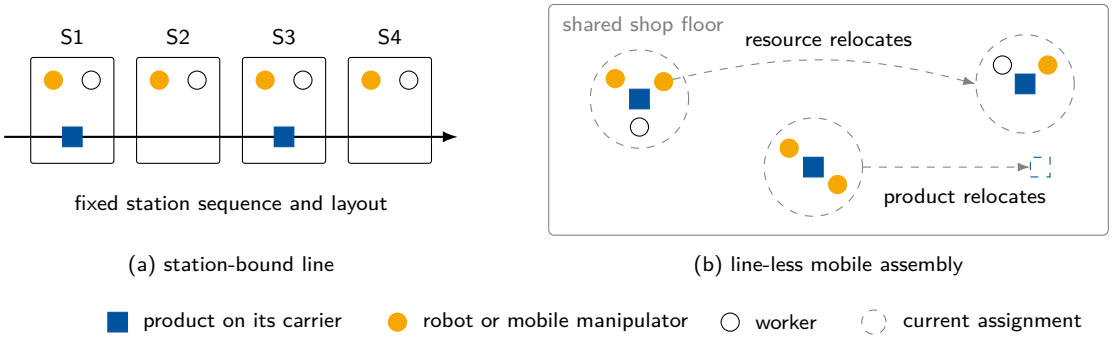


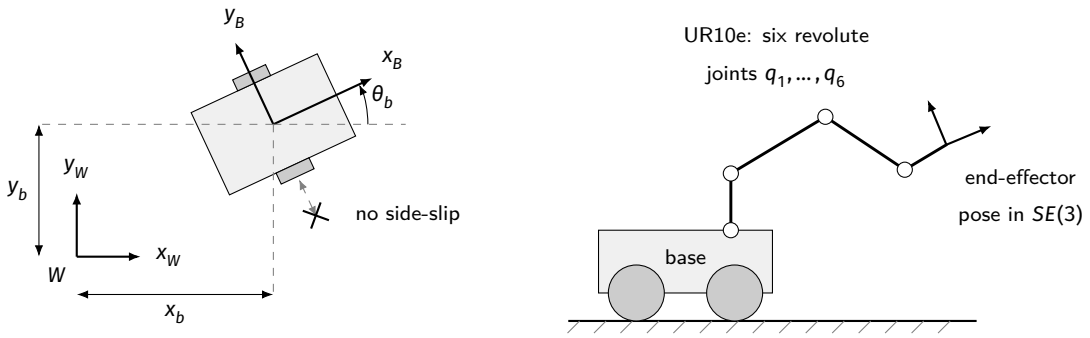
Figure 1.1: Station-bound assembly line and line-less mobile assembly system. (a) Every product passes the stations S1 to S4 in the same order, and each station keeps its own resources. (b) Products and resources are mobile entities on a shared shop floor; which resources serve which product, in what order and at which location is planned at run time. Schematic drawn after the descriptions in [HÜTT16; HÜTT19; BUCK19].

A central enabler of the LMAS concept is the *mobile manipulator*: a robotic arm mounted on a mobile base (Figure 1.2). The mobile base extends the reachable workspace far beyond the arm's own envelope and lets the robot reposition itself between and within tasks, while the arm provides the dexterity required for the assembly operation itself. Surveys of the field identify exactly this combination (one resource servicing many workstations, covering workpieces larger than a fixed-base arm could reach) as the motivation for wheeled mobile manipulation [SAND22; THAK23].

Adding a mobile base to the arm changes the motion-planning problem in three ways. In a classical line, an industrial arm operates in an engineered, static cell: the part arrives at a known pose, the surroundings are fixed, and the arm executes largely pre-programmed trajectories. In an LMAS, three of the assumptions behind that mode of operation are weakened.

Large, base-dependent workspace.

The target end-effector pose may lie outside the reach of the arm from the current base pose, so base and arm have to *cooperate* to accomplish a single task. Both mobile-manipulation surveys treat the choice of a base pose that renders the task reachable and well-conditioned as a planning problem in its own right [SAND22; THAK23].



(a) base pose in the plane

(b) arm mounted on the base

Figure 1.2: Wheeled mobile manipulator of the type studied here. (a) The base contributes the planar pose (x_b, y_b, θ_b); its wheels cannot slide sideways, which constrains the velocities it can produce but not the poses it can reach. (b) The arm contributes six revolute joints, and the task is the pose of the end-effector frame in $SE(3)$. Counting these groups against the task is what makes the system redundant (Equations (1.1) and (1.2)).

Shared workspace with moving resources.

Because resources are relocatable, parts of the environment move relative to one another during operation. How much of the environment is known in advance depends on the plant: a shop floor with tracked resources and a maintained world model is partially predictable, whereas human co-workers and manually moved carriers are not. The consequence for planning is therefore not that offline computation is impossible, but that a trajectory computed once for an assumed scene has to be revised while it is executed, which is the case that motivates online re-planning [FINE23].

Kinematic redundancy.

The base adds degrees of freedom (DOF) to a system that already has as many as the task requires. The arm is a spatial open chain with $N = 7$ links (including its mounting link) connected by $J = 6$

revolute joints, each contributing $f_i = 1$ freedom. Grübler's formula for a spatial mechanism ($m = 6$) gives its mobility [LYNC17]

$$\mathrm{dof} = m\,(N - 1 - J) + \sum_{i=1}^{J} f_i = 6\,(7 - 1 - 6) + 6 = 6. \tag{1.1}$$

The base contributes a planar pose, so the whole-body configuration is $q = (x_b, y_b, \theta_b, q_1, \ldots, q_6)$ and the configuration space is nine-dimensional, while the end-effector-pose task lives in the six-dimensional $SE(3)$:

$$\dim Q - \dim T = 9 - 6 = 3. \tag{1.2}$$

Three configuration-space directions therefore leave the end-effector pose unchanged. The picture differs at velocity level, because a differential-drive base cannot slide sideways: the rolling-without-slip condition $\dot{x}_b \sin\theta_b - \dot{y}_b \cos\theta_b = 0$ is a non-holonomic constraint, so the base offers only two instantaneous inputs (a forward and a turning rate) while still reaching all three planar coordinates over time [LYNC17; SICI09]. The instantaneous input vector is therefore eight-dimensional against a six-dimensional task twist, leaving two degrees of instantaneous redundancy. This redundancy is an opportunity: the same end-effector motion can be produced by infinitely many base–arm combinations, and the surplus can be spent on reach, on manipulability [YOSH85], or on obstacle avoidance [DIET15]. It is an opportunity only if the planner reasons about base and arm *jointly*; a planner that fixes the base first spends the redundancy before it can be used.

Together, these three conditions call for motion planning that is reactive, that spans the *combined* configuration space of base and arm, and that resolves the resulting redundancy deliberately rather than as a by-product. Whole-body motion planning, introduced next, addresses exactly this problem.

1.2 The Need for Whole-Body Motion Planning: From Sequential to Unified Approaches

The most common way to command a mobile manipulator is to treat base and arm as two independent subsystems and to plan for them *sequentially*: a navigation stack such as Nav2 [MACE20] first drives the base to a suitable stand-off pose; an arm-motion framework such as MoveIt [MoveIt 2] then solves the inverse kinematics for the target end-effector pose from that now-fixed base pose and plans a joint-space trajectory to the resulting configuration, typically with a sampling-based planner [ŞUCA12]. This decoupled scheme is modular and, above all, built from components that are mature and widely deployed in industry, which is the practical reason for its prevalence; both mobile-manipulation surveys report it as the dominant architecture in fielded systems [SAND22; THAK23].

Decoupling has three consequences that become severe in the LMAS setting; Figure 1.3 contrasts the two schemes.

Loss of reach and of solution quality.

Choosing the base pose *before* planning the arm discards the coupling between the two subsystems. A base pose that looks reasonable geometrically may leave the arm near a singularity or a joint limit, or may make the end-effector target unreachable, even though a jointly chosen base–arm configuration exists [SAND22; THAK23]. Both surveys describe base placement and arm planning as coupled sub-problems, and the holistic controller of Haviland et al. is motivated by the same observation [HAVI22].

Stop-and-go behaviour.

Sequential execution—drive, stop, then manipulate—prevents base and arm from moving *simultaneously*. Tasks in which the end-effector has to follow a continuous path longer than the arm's reach, which is a common LMAS case, cannot be executed smoothly this way [BURG24; PANK20]. Reported gains of on-the-move manipulation over the stop-and-go pipeline motivate both the reactive base controller of Burgess-Limerick et al. and the continuous mobile manipulation of Pankert and Hutter.

Cost of reacting to change.

A plan computed once, for a fixed base pose, does not by itself account for base drift, moving obstacles, or an updated target. This is a question of degree rather than of possibility: the pipeline can be re-triggered, and planners that predict how the scene will evolve and repair the trajectory online exist [FINE23]. What a decoupled pipeline cannot do cheaply is revise the *base* decision, because that decision sits upstream of the arm plan and invalidates it.

Whole-body motion planning removes this boundary by treating mobile base and arm as a *single* redundant kinematic (or dynamic) system and planning their motion in one optimisation [LYNC17; HAVI22]. The task is expressed directly in task space and the planner distributes the required motion between base and arm according to a chosen cost criterion, resolving the redundancy of Equation (1.2) explicitly rather than by construction. The formulation generalises redundancy resolution for kinematically redundant manipulators [SICI09] and null-space projection for a fixed base [DIET15] to base and arm together. This addresses the three points above: the full combined workspace becomes available, base and arm move simultaneously along one coordinated trajectory, and the base decision is revisited at every re-solve instead of being frozen upstream. How well the last property holds in

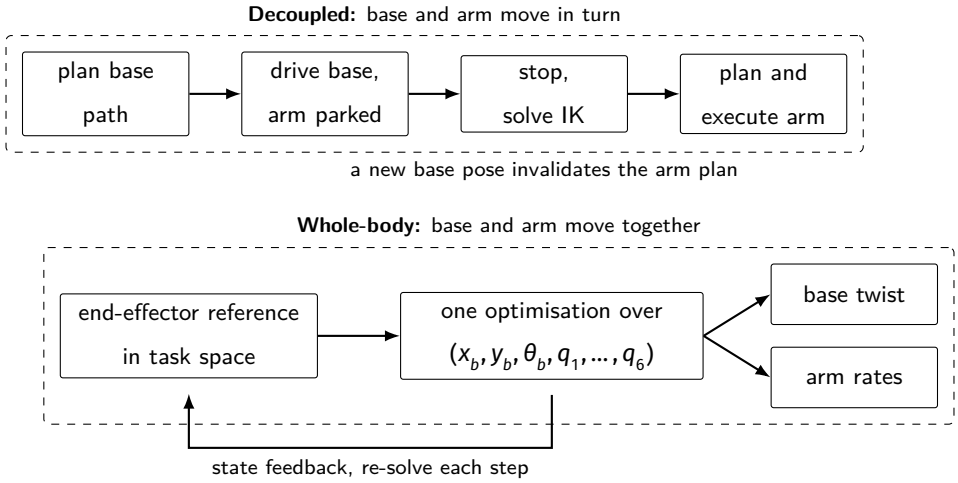


Figure 1.3: Decoupled versus whole-body command of a mobile manipulator. The decoupled pipeline fixes the base pose upstream of the arm plan, so the two subsystems act in sequence. A whole-body scheme optimises over the joint configuration, emits base and arm commands from the same solution, and closes the loop on the measured state.

practice depends on how fast the optimisation can be re-solved relative to the rate at which the scene changes; this work quantifies that trade-off on hardware in Chapter 5.

Formulating whole-body planning as a receding-horizon optimal-control problem, that is, as *Model Predictive Control* (MPC) [RAWL17], suits mobile manipulation for two reasons. First, the constraints that make the problem hard—joint and velocity limits, self-collision, and the non-holonomic condition of a wheeled base—are inequality constraints on states and inputs, which is the form an MPC formulation handles natively [MAYN00]. Second, the solver re-plans from the measured state at every step, so the same mechanism that produces the plan also provides the feedback. Whole-body MPC on this basis has been demonstrated on a dynamically stable mobile manipulator [MINN19] and for continuous mobile manipulation from onboard perception [PANK20]. The open-source OCS2 (Optimal Control for Switched Systems) framework [OCS2] provides an implementation of this approach and has been used for whole-body loco-manipulation on legged and wheeled platforms [SLEI21].

1.3 Objectives and Structure of This Work

1.3.1 Objective

The goal of this work is to *integrate and validate* whole-body MPC, based on the OCS2 framework, on the HERo mobile manipulator (a differential-drive mobile base carrying a Universal Robots UR10e arm [UR10e]), and to characterise the resulting end-effector-pose tracking behaviour under realistic

conditions. In contrast to studies that remain in simulation, the emphasis is on a *deployable* system: the MPC runs in a distributed, real-time software architecture on the physical robot, and its performance is assessed on hardware.

1.3.2 Requirements on the Motion Planner

The discussion above yields the criteria that any candidate planner has to satisfy on HERo. They are stated here explicitly, so that the comparison in Section 2.3 has a fixed reference and does not rest on an implicit notion of suitability. Table 1.1 lists them.

Table 1.1: Requirements on the motion planner for HERo, with the argument each one follows from. Table 2.1 scores the candidate approaches against exactly these criteria.

ID	Requirement	Origin
R1	Coordinated base–arm motion	Reach and stop-and-go arguments, Section 1.2
R2	Explicit constraint handling	Joint, velocity, non-holonomic and self-collision limits, Equation (1.2) and Section 1.2
R3	Deliberate redundancy resolution	Three surplus configuration directions, Equation (1.2)
R4	Online re-planning from the measured state	Moving resources in a shared workspace, Section 1.1
R5	Real-time feasibility on the on-board computer	Deployment on HERo rather than in simulation, Section 1.3.1
R6	ROS 2 interfaces	Integration with the existing HERo software stack, Chapter 3
R7	Transparency and analysability	Behaviour has to be diagnosable on hardware, Chapter 5

1.3.3 Scope and Contributions

The concrete contributions of this thesis are:

1. the selection of a suitable whole-body motion-planning approach for the HERo platform, following a comparison of candidate frameworks against Table 1.1 (Chapter 2);

2. the integration of OCS2 whole-body MPC into the HERo software stack, including a distributed host/on-board architecture for real-time operation, the whole-body model and its parameterisation, and the hardware and ROS-control interfaces (Chapter 3);

3. a simulation-based validation of the closed loop that isolates the geometry, timing, and data pipeline before hardware deployment, and that measures the residual the optimiser leaves on its own model (Chapter 4); and

4. the deployment of the system on the physical HERo platform, together with a discussion of the deployment challenges encountered and preliminary end-effector-tracking results (Chapter 5).

1.3.4 Structure

Chapter 2 reviews the state of the art in whole-body motion planning for mobile manipulators, classifies the main approaches, presents OCS2 and alternative frameworks, and justifies the selection made here. Chapter 3 describes the HERo platform and the integration of OCS2 into it: the distributed architecture, the whole-body model and its parameterisation, and the interfaces to the low-level control layer. Chapter 4 presents the simulation-based validation using a kinematic closed-loop (dummy MRT) setup. Chapter 5 reports on the hardware deployment, the challenges and their mitigations, and preliminary end-effector-tracking results. Chapter 6 summarises the work, reflects critically on its limitations, and gives recommendations for resolving the remaining issues.

2

State of the Art in Whole-Body Motion Planning for Mobile Manipulators

Chapter 1 argues that the Line-less Mobile Assembly System (LMAS) setting calls for whole-body motion planning rather than a decoupled base–arm pipeline, and states the resulting requirements R1 to R7 in Table 1.1. This chapter surveys how the literature approaches the problem and narrows the field down to the framework used in the remainder of this thesis. Section 2.1 introduces a classification of motion-planning approaches for mobile manipulators. Section 2.2 then focuses on *holistic* (whole-body) planners, presenting the OCS2 framework (Section 2.2.1) and the main alternatives (Section 2.2.2). Section 2.3 scores the candidates against Table 1.1 and justifies the selection of OCS2.

2.1 Classification of Motion Planning Approaches

Motion planning for mobile manipulators can be organised along a few largely orthogonal axes. The two published reviews of the field distinguish approaches primarily by *how base and arm are coupled* and by *the algorithmic principle* used to generate motion [SAND22; THAK23]; a third, practical axis concerns *when the computation happens*. Figure 2.1 shows how the axes overlap and where the families discussed below fall.

Decoupled versus whole-body.

The first and, for this work, most important distinction is between *decoupled* and *whole-body* schemes. A decoupled scheme plans base and arm separately and in sequence: a navigation stack drives the base to a stand-off pose, after which an arm planner solves the manipulation task from that fixed base configuration. A whole-body scheme instead treats base and arm as a *single* redundant kinematic (or dynamic) system and solves for their motion jointly, so that the task-space objective is distributed

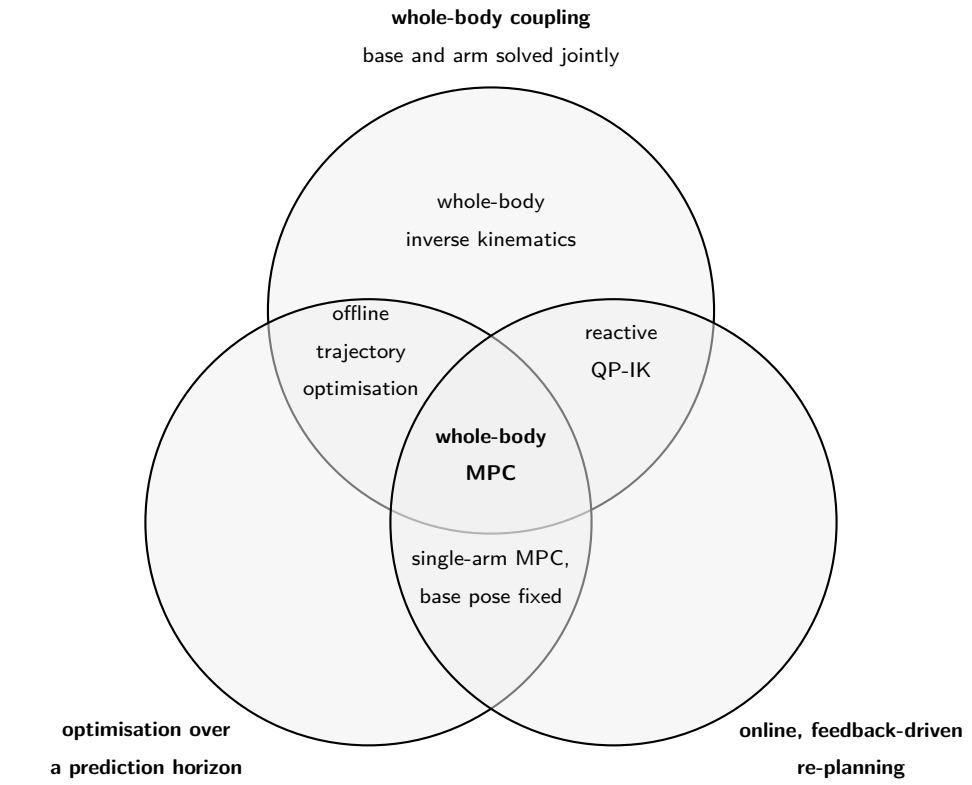


Figure 2.1: The three classification axes and the approaches that fall in each region. Whole-body MPC is the only family in the intersection of all three, which is the region Table 1.1 asks for. Sampling-based arm planning sits outside: it is decoupled, computed before execution, and optimises no running cost.

between base and arm by the planner itself. This is the formulation used by the holistic reactive controller of Haviland et al. [HAVI22] and by the whole-body MPC of Minniti et al. [MINN19]; it generalises null-space redundancy resolution from a fixed base [DIET15] to the combined system. As set out in Section 1.2, decoupling sacrifices reach, solution quality, and the ability to revise the base decision, at the price of a higher-dimensional and more tightly coupled problem.

Algorithmic principle.

Independently of the coupling, the motion itself can be produced by different categories of algorithm.

- **Sampling-based planning** explores the configuration space by randomly sampling and connecting collision-free states, as in Probabilistic Roadmaps [KAVR96] and Rapidly-exploring Random Trees [LAVA01], both of which are exposed through OMPL and, in turn, through MoveIt [ŞUCA12; MoveIt 2]. These methods are effective at finding feasible paths in cluttered, high-dimensional spaces and are probabilistically complete, but the path they return is feasible

rather than optimal with respect to a running cost, and the computation is normally performed before execution rather than inside a control loop [LAVA06].

- **Optimisation-based planning** formulates motion as a constrained optimisation over a trajectory. Solving that optimisation once, before execution, over the complete trajectory is *trajectory optimisation*; re-solving it online over a receding horizon and applying only its first part turns the same formulation into *Model Predictive Control* (MPC) [RAWL17; MAYN00]. Both variants handle joint, velocity, and collision limits as explicit constraints and return smooth motion that is optimal with respect to the stated cost.
- **Reactive schemes** compute an instantaneous command from the current state, for example through potential fields [KHAT86] or through a quadratic program solved once per control step (QP-IK) [HAVI21]. A single quadratic program is cheaper to solve than an optimisation over a horizon of the same model, which is why these schemes run at control rate on modest hardware; the price is that they optimise only the current instant and therefore cannot trade a worse command now for a feasible one later.

Offline versus online.

Finally, a planner may operate *offline*, computing a complete trajectory before execution, or *online*, re-planning continuously as new state estimates and targets arrive. It is the online, feedback-driven variant that supplies the disturbance rejection required in a shared, changing LMAS workspace, because an offline trajectory carries no mechanism for reacting to a scene that has moved since it was computed [FINE23]. Whole-body MPC sits at the intersection of the *whole-body*, *optimisation-based*, and *online* categories (Figure 2.1), which is the region of the design space this thesis targets.

2.2 Overview of Holistic Motion Planners

Holistic (whole-body) planners solve for base and arm motion in a single formulation. The mature, deployable representatives of this class are predominantly optimisation-based, because expressing the task as a cost and the physical restrictions as constraints maps directly onto the coupled, redundant, and constrained nature of the mobile-manipulation problem. Among open-source stacks that couple a navigation stack with a genuine whole-body controller, three integration mechanisms recur: replacing the Nav2 local planner through the `nav2_core::Controller` plugin interface, wrapping the whole-body controller in `ros2_control` as `wbc_ros` does [wbc_ros; ros2_control], and bridging the navigation stack's velocity and goal topics to a whole-body solver while the robot state reaches the solver through the joint-state publisher, as in the WoLF stack built on OCS2 [WoLF]. On the algorithmic side the same set of stacks is dominated by two formulations: optimal control

over a horizon [SLEI21; PANK20] and constraint-based task-space control solved as a quadratic program [STEL22; STUE21].

Figure 2.2 recalls how the receding-horizon mechanism at the core of the first formulation works. The following subsections describe OCS2 and then position the principal alternatives against it.

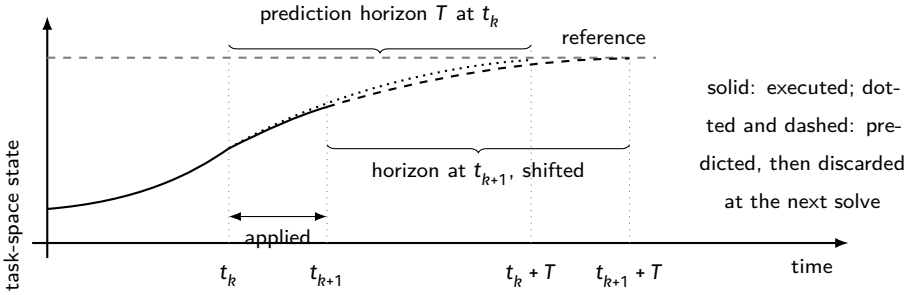


Figure 2.2: Receding-horizon principle. At t_k the solver optimises the whole trajectory over the horizon T but only the first segment is applied; at t_{k+1} the horizon shifts, the problem is re-solved from the newly measured state, and the remainder of the previous prediction is discarded. Re-solving from the measurement is what turns the plan into feedback [RAWL17].

2.2.1 The OCS2 Framework: Optimal-Control-Based Model Predictive Control

OCS2 (*Optimal Control for Switched Systems*) is an open-source library written in C++ for numerical optimal control and developed at the Robotic Systems Lab, ETH Zürich. It provides solvers for continuous-time optimal-control problems, most notably Sequential Linear-Quadratic (SLQ) control, a constrained variant of differential dynamic programming, together with the infrastructure required to run them as an MPC controller; the solver was introduced for quadrupedal locomotion and later extended with a feedback policy and with relaxed barrier functions for constraints [FARS17; GRAN19; FELL17; OCS2].

Three properties make OCS2 suitable for whole-body mobile manipulation; Figure 2.3 shows how they fit together. First, the derivatives of cost and constraints that the solver needs are obtained by *automatic differentiation*: the model (kinematics, cost terms, constraints) is supplied symbolically, and OCS2 generates and compiles the corresponding derivative code through CppAD/CppADCodeGen, so that a change of model does not require hand-derived Jacobians. Second, a *reference manager* allows the desired task-space trajectory to be replaced while the solver is running, which is the interface a target-generating node needs in order to steer the controller. Third, and decisive for a real-time deployment, OCS2 separates optimisation from execution through an MPC–MRT split: the MPC process solves the optimal-control problem at its own rate and publishes a *policy*, while a lightweight Model Reference Tracking (MRT) client evaluates that policy at the control rate to produce the instantaneous command. The split makes it possible to run optimiser and hardware

interface as separate processes, even on separate machines, a property this thesis exploits in the distributed architecture of Chapter 3.

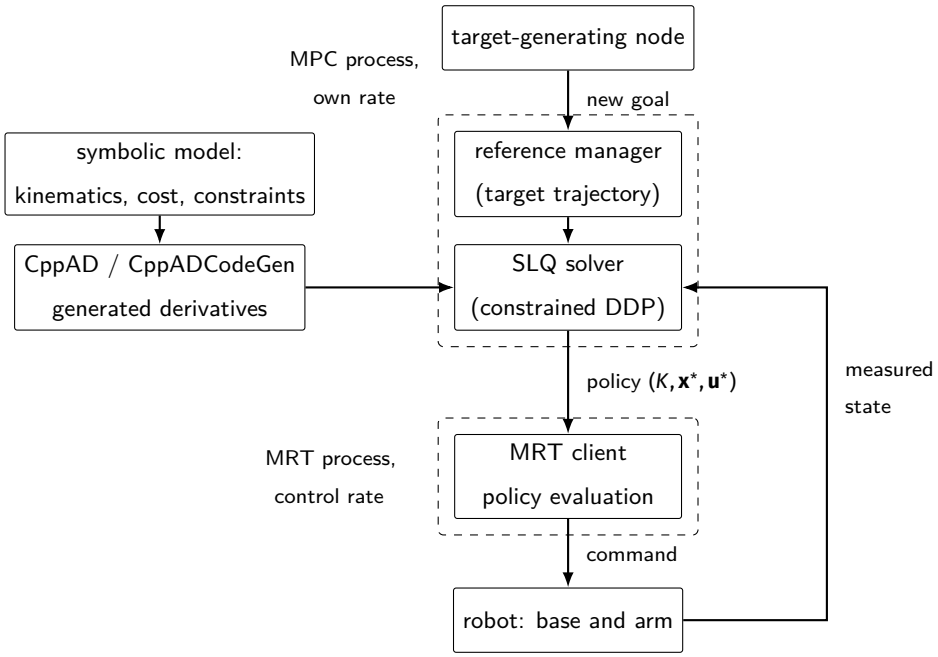


Figure 2.3: Structure of OCS2. The symbolic model is differentiated automatically and compiled once; the SLQ solver runs in its own process at its own rate and publishes a policy; the MRT client evaluates that policy at control rate and closes the loop on the robot. The reference manager lets the task-space target be replaced without restarting the solver.

OCS2 has been used for whole-body mobile manipulation across several platforms. The kinematic planner of Giftthaler et al. addresses exactly the non-holonomic base considered here [GIFT17]; Minniti et al. demonstrate whole-body MPC on a dynamically stable mobile manipulator [MINN19] and extend it to force-controlled interaction with the environment [MINN21]; Pankert and Hutter run continuous mobile manipulation from onboard perception [PANK20]; Sleiman et al. unify locomotion and manipulation in one MPC formulation [SLEI21]; and Mittal et al. apply the same toolbox to articulated objects in unknown scenes [MITT22]. This record covers R1 to R4 of Table 1.1 on hardware of comparable complexity, and the process-separable architecture speaks to R5, which is why OCS2 is carried forward as the leading candidate for HERo.

2.2.2 Alternative Frameworks and Methods

Several alternatives to OCS2 exist, spanning the algorithmic categories of Section 2.1.

Sampling-based planning (MoveIt).

MoveIt [MoveIt 2] is the de-facto standard motion-planning framework in the ROS ecosystem, exposing the OMPL sampling-based planners [ŞUCA12]. Its strength is collision-free planning for redundant arms in cluttered scenes, computed before execution; it can in principle be extended to a mobile base by treating the base pose as additional planning variables. In its open-source form, however, MoveIt underpins the *decoupled* navigate-then-manipulate pipeline and provides no online, coordinated base–arm feedback of the kind R1 and R4 ask for. Whole-body capabilities for mobile manipulators are offered by the commercial MoveIt Pro platform, which is not open source and therefore does not satisfy R7 [MoveIt Pro]. MoveIt is consequently the incumbent pipeline from which this work departs rather than a competing whole-body planner; a quantitative comparison of the two pipelines requires a second, decoupled implementation on the same platform and is left to future work (Chapter 6).

Other optimisation-based and MPC tools.

Beyond OCS2, whole-body behaviour can be realised with constraint-based task-space controllers driven by a solver over a short horizon. Stelter et al. publish an open-source framework for mobile manipulators that casts the task-space constraints as a linear MPC problem [STEL22], and Stueben et al. apply constraint-based whole-body control to mobile manipulators in human-centred environments [STUE21]. The `wbc_ros` package wraps a quadratic-program whole-body controller in `ros2_control` [wbc_ros]. These tools satisfy R1 and R2 but either target a different base morphology or expose a shorter effective horizon than a full optimal-control formulation, and the integration effort for a differential-drive base under ROS 2 is comparable to the one reported here.

Reactive quadratic-program inverse kinematics.

QP-based inverse kinematics resolves the end-effector task and secondary objectives—joint-limit avoidance, manipulability—into joint velocities at each control step, and extends naturally to base and arm together [HAVI21; HAVI22]. Burgess-Limerick et al. show that this class of controller supports manipulation while the base is moving [BURG24], so it meets R1, R3, and R4. Because it optimises a single step, however, it cannot anticipate a constraint that will become active later in the motion, which is what R2 asks for over a horizon.

2.3 Comparison of Motion Planning Algorithms and Selection of a Suitable Algorithm

The choice of a planner for HERo follows the requirements R1 to R7 stated in Table 1.1: coordinated whole-body motion, explicit handling of joint, velocity, non-holonomic and self-collision constraints, deliberate redundancy resolution, online re-planning from the measured state, real-time feasibility on the on-board computer, ROS 2 interfaces, and a controller whose behaviour can be analysed on hardware. Table 2.1 scores the three candidate families against those criteria. The scores are judgements, not measurements: each one is read off the published evidence cited in Sections 2.2 and 2.2.2 for that family, and no row rests on a trial run of the alternatives on HERo. Only the OCS2 column is later confirmed by measurement, in Chapter 5.

Table 2.1: Qualitative comparison of motion-planning approaches against the requirements of Table 1.1 (+ good, ◦ partial, – weak). Scores follow the evidence cited per family in Sections 2.2 and 2.2.2.

	Criterion	Sampling (MoveIt)	QP-IK (reactive)	Whole-body MPC (OCS2)
R1	Coordinated whole-body motion	◦	+	+
R2	Explicit constraint handling	◦	◦	+
R3	Redundancy resolution	◦	+	+
R4	Online re-planning	–	+	+
R5	Real-time feasibility	◦	+	◦
R6	ROS 2 interfaces	+	◦	+
R7	Transparency / analysability	+	+	+

Sampling-based planning is mature and well supported but computes its trajectory before execution and, in its standard form, keeps base and arm separate, which disqualifies it as the primary whole-body planner while keeping it relevant as the incumbent alternative. Reactive QP-IK covers the coordination, redundancy and reactivity requirements R1, R3 and R4, and, being a single-step program, it is the lightest of the three to evaluate inside a control loop (R5); it stays partial on R2, because such a program cannot anticipate a constraint that becomes active later in the motion, and on R6. Whole-body MPC is the only one of the three that Table 2.1 scores + on R1 to R4 together; its weak point is R5, the computational burden of solving an optimal-control problem online, which is addressed through the MPC–MRT split and a distributed deployment (Chapter 3) and quantified experimentally in Chapter 5.

Among whole-body MPC implementations, OCS2 is selected for its open-source implementation, its explicit constraint support, its demonstrated use on wheeled and legged mobile manipulators (Section 2.2.1), and, decisive for a real-time deployment on distributed hardware, its process-separable MPC–MRT architecture. The remainder of this thesis therefore integrates and validates OCS2-based whole-body MPC on the HERo platform.

3

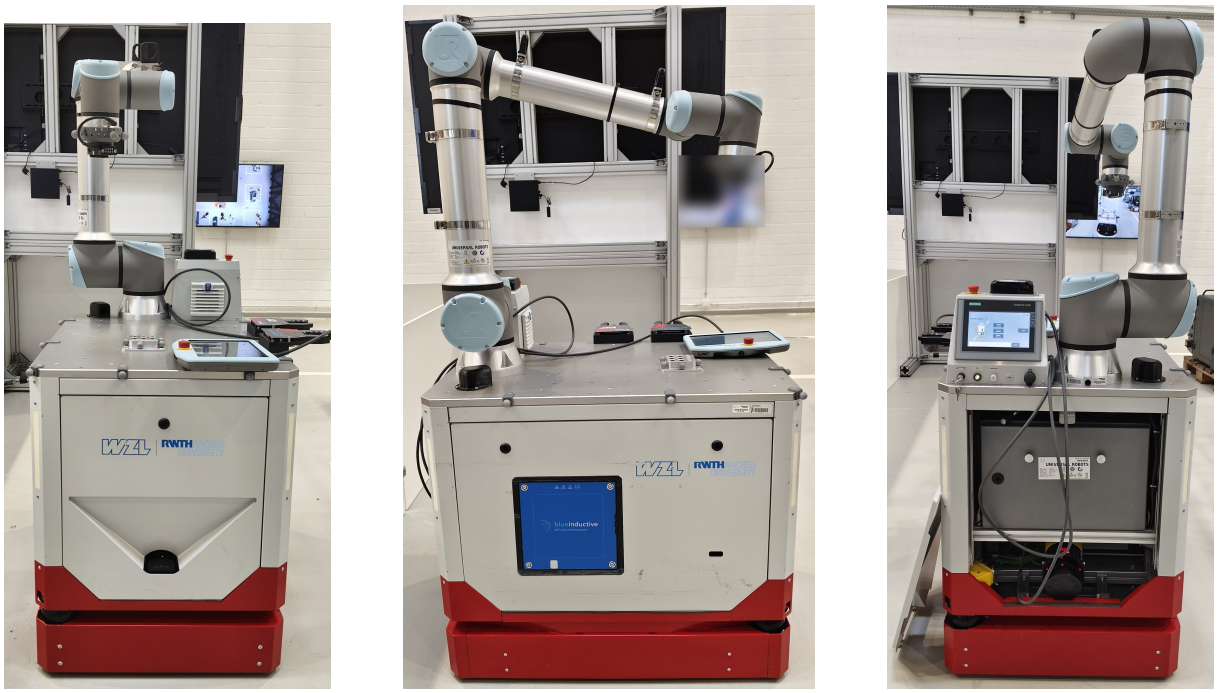
Integration of OCS2 into the HERo Mobile Manipulator

This chapter documents the system that was built to run OCS2 whole-body MPC on the physical HERo platform. Section 3.1 describes the platform and the properties that constrain the controller. Section 3.2 presents the two-machine deployment that makes the optimisation affordable at run time, and Section 3.3 the whole-body model together with the operating point it was tuned to. Section 3.4 describes the layer that turns an OCS2 policy into wheel and joint commands. Unless stated otherwise, parameter values are those of the on-robot configuration file `task_hardware.info`; measured timing and tracking results are reported in Chapter 5.

3.1 HERo Platform: Differential-Drive Base and UR10e Manipulator

The HERo platform combines a HENKEL + ROTH *freeMoVe* transport base (product designation *mobile robot mr 3.0*) with a Universal Robots UR10e six-axis arm [HENK19; HERT20; UR10e]. The chassis measures 0.95 m × 0.65 m × 0.815 m and carries a modelled mass of 400 kg in the kinematic description used throughout this work. Two independently actuated wheels of radius 0.08 m at a track width of 0.548 m provide the drive; four passive casters carry the remaining load. The wheel drives are commanded over a CANopen bus. The arm is mounted 0.830 m above the ground on a plate offset by -0.355 m in x and -0.050 m in y from the base origin and rotated by $-90°$ about the vertical axis, so that the shoulder sits roughly at the height of a workpiece carrier. Figure 3.1 shows the platform from three sides.

Two properties of this combination shape the control problem. The first is the drive geometry: the base pose has three planar degrees of freedom but only two independent velocity inputs, and the base cannot translate sideways [LYNC17]. A whole-body planner must respect this non-holonomic restriction, otherwise it produces end-effector paths that the base cannot follow [GIFT17]. The



(a) Front **(b)** Side **(c)** Rear

Figure 3.1: The HERo platform: UR10e arm on the *freeMoVe* base with its red drive unit. The teach pendant lies on the top plate in (a) and (b). The operator panel next to the shoulder in (c) is the `hmi` link of the kinematic description; the UR control box sits inside the chassis below it.

second is inertia. Moving a 400 kg platform is slow, and the safety concept of the platform limits its speed independently of what a controller may request, which is why the base velocities in Section 3.3 are capped an order of magnitude below the arm's capability.

The kinematic tree is rooted at the frame `footprint`, which lies on the ground plane midway between the drive wheels, with x forward, y to the left and z upwards. The controlled end-effector frame is `tool0`, the flange frame that coincides with the tool orientation shown on the teach pendant; the arm's own `base_link` is not used, because its axes are rotated with respect to the pendant convention and would invite sign errors when comparing commanded and measured poses.

Figure 3.2 shows the tree, the frames it defines and the state components attached to them.

Figure 3.2 is a reduction. The transform tree that the running system publishes has 51 frames and is shown in full in Figure 3.3. Ten of its 50 transforms change over time: the base pose, the six arm joints, the two wheel joints, and the pose of the end-effector target, which the OCS2 visualisation adds for display. All other frames are attachments, sensors and tool frames that are rigidly fixed to one of the frames on the trunk.

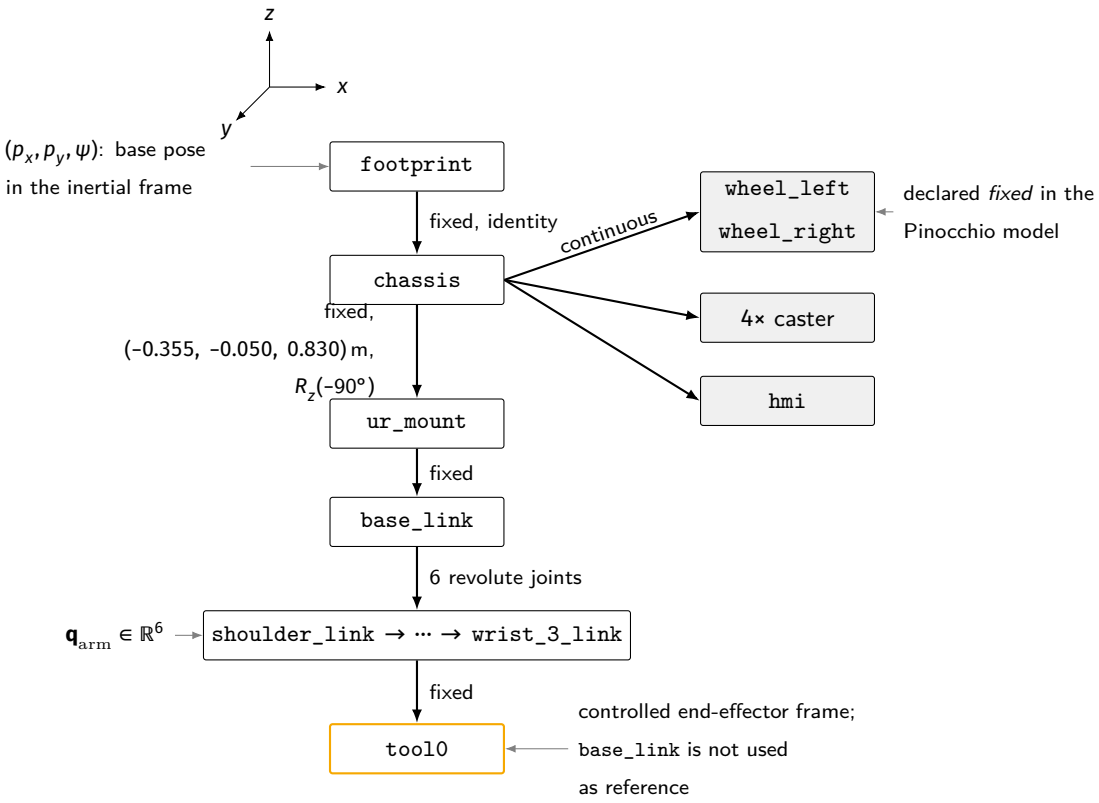


Figure 3.2: Kinematic tree of HERo as the Pinocchio model sees it, with the transforms read from the platform description. The frame `footprint` lies on the ground plane midway between the drive wheels, at a track of 0.548 m and a wheel radius of 0.08 m. Shaded links carry no optimised degree of freedom: the two wheel joints are declared fixed, because the wheels are not part of the chain from base to end-effector, and the casters and the operator panel are fixed in the description itself. The nine state components of Equation (3.1) are attached to `footprint` and to the six revolute arm joints.

Beyond the drives, the platform carries five SICK safety laser scanners, two safety controllers, wheel brakes, and an operator panel. None of these enters the MPC loop. The controller described here consumes exactly two measurements, wheel odometry and arm joint positions, and relies on the platform's own safety layer for protective stops. Obstacle avoidance with respect to the environment is consequently not part of the optimisation; only self-collision between the arm and the platform is (Section 3.3).

Computation is split over two machines on the robot's own `192.168.12.0/24` subnet. An industrial PC (IPC) inside the platform hosts the ROS 2 Jazzy control stack in a container and reaches the arm controller over the same subnet; an external PC joins the subnet either by cable or through the platform's wireless access point [ROS 2; MACE22]. Which software runs where is the subject of the next section.

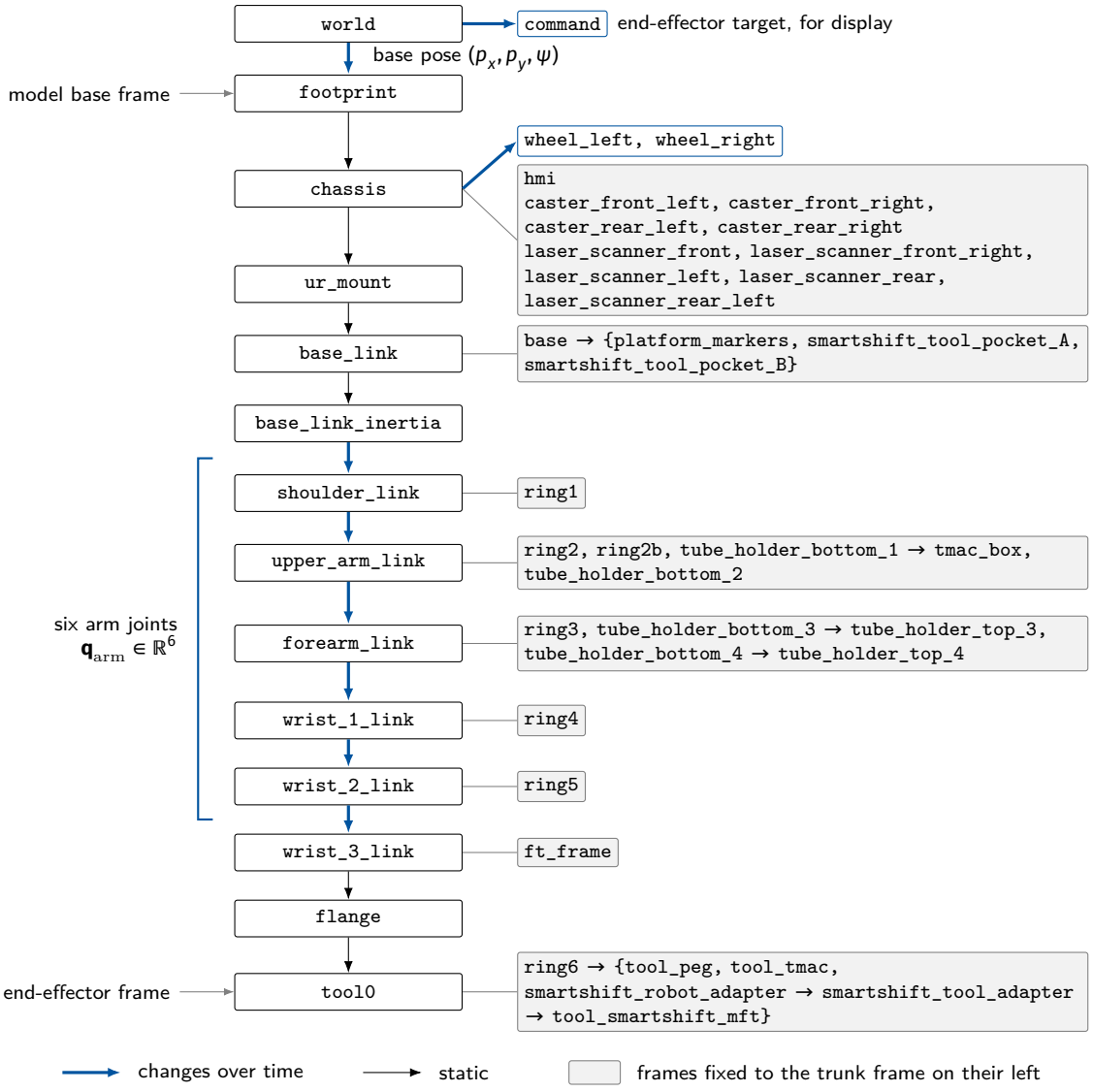


Figure 3.3: Complete transform tree of HERo, redrawn from a `view_frames` recording taken while the OCS2 visualisation was running. The trunk runs from `world` to the end-effector frame `tool0`. Thick blue arrows are the ten transforms that change over time; every other transform is static. A grey box lists the frames attached to the trunk frame on its left, and an arrow inside a box leads to a further child. The operator panel `hmi`, the tube holders and the marker rings `ring2` to `ring6` reappear among the bodies of the self-collision model in Section 3.3.

3.2 Distributed System Architecture for Real-Time MPC

Solving the optimal-control problem is the most expensive part of the loop relative to the remaining nodes on the IPC, and the IPC is the smaller of the two machines available. It carries an Intel Core i5-6300U with two cores and four threads and 7.7 GB of memory, runs a kernel without real-time patches, and keeps the CPU frequency governor at `powersave`; the external PC carries an AMD Ryzen 7 7840HS with eight cores and sixteen threads. The solver alone is configured for eight threads (Table 3.3), twice what the IPC offers before the control chain takes its share, so running it on board would have forced a shorter horizon or a coarser model.

The MPC–MRT split of OCS2 (Section 2.2.1) removes that constraint: the optimiser and the client that evaluates its policy are separate processes that exchange messages, so they may live on different machines [OCS2].

The deployment used here puts them on either side of the machine boundary. The external PC runs the MPC node, the target-generating node and the visualisation, while the IPC runs the low-level control chain and the MRT bridge. The fast path—measure, evaluate the policy, actuate—is then entirely local to the IPC, and only three OCS2 streams cross the machine boundary. Table 3.1 lists what runs where and what the network carries.

Table 3.1: Process placement and the traffic that crosses the external PC–IPC boundary. Keeping the MRT bridge on the IPC is what leaves the actuation path machine-local.

External PC	MPC node, target node, RViz
IPC	control chain, MRT bridge
Crossing the network	`_mpc_observation` (IPC → external PC, 50 Hz), `_mpc_policy` (external PC → IPC), `_mpc_reset` service at start-up
Effect of a link outage	the bridge keeps evaluating the last policy locally until its validity window expires, then holds
Clock synchronisation	not required

The last of those rows follows from where wall-clock times are compared. OCS2 timestamps observations and policies on a relative time axis whose origin the bridge fixes at the first accepted measurement, so the MPC–MRT exchange itself is insensitive to an offset between the two machine clocks. The bridge does compare wall-clock times in one place, its staleness check on incoming measurements, and there the two operands share the IPC clock: the bridge runs on the IPC, and odometry and joint states are published by the control chain on the same machine. An offset to the

external PC therefore never enters the 0.2 s staleness threshold, and a time service such as Chrony is not needed to keep the loop running [chrony]. Chrony does run on the IPC, with the external PC as its reference; at the time of writing it reported an offset of −0.7 ms at an RMS of 1.5 ms. That keeps logs recorded on the two machines comparable, but the control loop does not depend on it.

The deployment runs with the Zenoh middleware implementation rather than the default DDS transport of ROS 2. The switch was made during bring-up, after DDS discovery over the platform's wireless link had repeatedly failed to establish the OCS2 streams; no side-by-side comparison of the two middlewares was recorded, so this is a deployment decision and not a measured result [Zenoh; MACE22]. Zenoh requires a router process on each machine, with one router configured to connect to the other; the external PC router is pointed at the IPC router. Figure 3.4 shows the resulting node and topic layout.

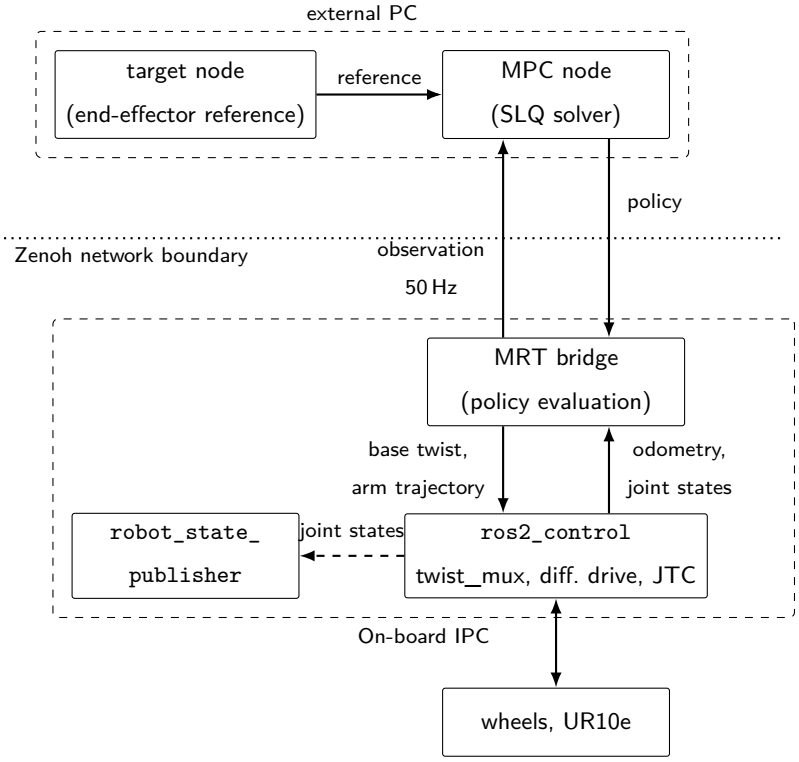


Figure 3.4: Deployment of OCS2 on HERo. The external PC solves the optimal-control problem; the IPC evaluates the resulting policy and actuates the robot, so the loop that runs at 50 Hz is machine-local. The reset service and the visualisation on the external PC are omitted.

One consistency requirement is easy to violate and is stated here explicitly. The MPC node and the MRT bridge each build their own Pinocchio model from a URDF file that is exported from the platform description at launch time [CARP19]. The two exports must use the same end-effector variant and the same kinematic calibration file, otherwise the solver optimises for one robot and the bridge evaluates the policy on another. The visualisation, in contrast, takes its model from

the `robot_state_publisher` on the IPC, which is the single authoritative source of `/tf` and `/robot_description`; the external PC launch deliberately starts no second publisher. Because OCS2 hardcodes `world` as its inertial frame while the robot tree is rooted at `odom`, an identity transform between the two is published on the external PC. The base state fed to the solver is wheel odometry, so `world` and `odom` coincide numerically; the price is that the controller inherits the drift of the odometry, and no external localisation corrects it.

Start-up follows a fixed order: the Zenoh routers first, then the MPC node on the external PC, then the IPC. The MPC node advertises the reset service that the bridge needs for its initial handshake, so starting it first avoids a wait. The reverse order also converges, because the bridge waits for the service rather than aborting (Section 3.4). On the first start after a model change, OCS2 generates and compiles the derivative code before the solver runs, which adds a delay of roughly one minute; the compiled libraries are then cached on disk.

3.3 Whole-Body Model and Parameterisation

OCS2 ships a mobile-manipulator model family, of which the *wheel-based* variant matches HERo: a non-holonomic planar base carrying a serial arm [OCS2]. The Pinocchio model is built from the platform URDF with the two wheel joints declared fixed, since the wheels are not part of the kinematic chain from base to end-effector and their rotation is irrelevant to the task [CARP19]. The base frame is `footprint` and the end-effector frame is `tool0`, as introduced in Section 3.1. The resulting state and input are

$$\mathbf{x} = \left(p_x,\ p_y,\ \psi,\ \mathbf{q}_{\mathrm{arm}}\right) \in \mathbb{R}^9, \qquad \mathbf{u} = \left(v,\ \omega,\ \dot{\mathbf{q}}_{\mathrm{arm}}\right) \in \mathbb{R}^8, \tag{3.1}$$

with (p_x, p_y, ψ) the base pose in the inertial frame, $\mathbf{q}_{\mathrm{arm}} \in \mathbb{R}^6$ the arm joint positions, v and ω the base forward and yaw rate, and $\dot{\mathbf{q}}_{\mathrm{arm}}$ the arm joint velocities. The system equation is the differential-drive kinematics augmented by the arm joints [LYNC17],

$$\dot{\mathbf{x}} \;=\; f(\mathbf{x}, \mathbf{u}) \;=\; \begin{pmatrix} v\cos\psi \\ v\sin\psi \\ \omega \\ \dot{\mathbf{q}}_{\mathrm{arm}} \end{pmatrix}, \tag{3.2}$$

which encodes the non-holonomic restriction of Section 3.1: no input produces motion along the base y-axis. The model is kinematic, not dynamic. The control chain used here issues only a twist to the drive controller and position setpoints, at a fixed rate, to the joint-trajectory controller. No

validated inertial parameters exist for the loaded platform either. A dynamic model would therefore add eight velocity states (v, ω and the six arm joint rates) and an unverified mass matrix, while the commands that reach the actuators would remain twists and joint positions. The cost of this choice is that inertial effects appear to the solver only through the velocity limits.

The objective combines an input cost with soft constraints. Over a horizon T starting at t_0,

$$J = \int_{t_0}^{t_0+T} \Big[\tfrac{1}{2}\,\mathbf{u}^{\mathsf{T}} R\,\mathbf{u} + \tfrac{1}{2}\,\mu_{\mathrm{p}} \|e_{\mathrm{p}}(\mathbf{x})\|^2 + \tfrac{1}{2}\,\mu_{\mathrm{o}} \|e_{\mathrm{o}}(\mathbf{x})\|^2 + B_{\lim}(\mathbf{x},\mathbf{u}) + B_{\mathrm{col}}(\mathbf{x}) \Big]\,\mathrm{d}t \;+\; \Phi\big(\mathbf{x}(t_0+T)\big), \tag{3.3}$$

where e_{p} is the end-effector position error and e_{o} the orientation error, both taken with respect to the reference pose interpolated from the target trajectory at the current time, and Φ repeats the two end-effector terms at the end of the horizon. The reference is supplied online through the OCS2 reference manager, either by an interactive marker in RViz or by the waypoint node used in Chapters 4 and 5. OCS2 interpolates that reference linearly and component-wise between successive knots, the quaternion included, which is why the waypoint node commands one constant orientation for the whole path (Section 4.2.1). Figure 3.5 collects the terms and the quantities each of them acts on.

The input weight is diagonal and separates base from arm, $R = \mathrm{diag}(R_{\mathrm{base}}, R_{\mathrm{arm}})$ with $R_{\mathrm{base}} = 0.085 \cdot I_2$ and $R_{\mathrm{arm}} = 0.04 \cdot I_6$. The ratio is the tuning knob that distributes a commanded end-effector motion between base and arm: base motion is charged roughly twice as much as arm motion, which keeps the heavy platform still for targets the arm can reach on its own and lets it contribute once the arm approaches its reach limit. Chapter 5 returns to this ratio, because it also determines whether the base starts moving early enough for paths that leave the arm's workspace.

Limits and self-collision enter as relaxed barrier penalties, the OCS2 treatment of an inequality $h(\mathbf{x},\mathbf{u}) \geq 0$ [FELL17; OCS2]:

$$B(h) = \begin{cases} -\mu \ln h, & h > \delta, \\ -\mu \ln \delta + \dfrac{\mu}{2}\left[\left(\dfrac{h-2\delta}{\delta}\right)^2 - 1\right], & h \leq \delta. \end{cases} \tag{3.4}$$

The barrier is logarithmic while the constraint is comfortably satisfied and switches to a quadratic extension below the relaxation width δ, which keeps the cost and its derivatives finite when a constraint is violated—necessary here, because measured states from the real robot do occasionally violate a limit. Figure 3.6 shows the penalty at the configured relaxation width.

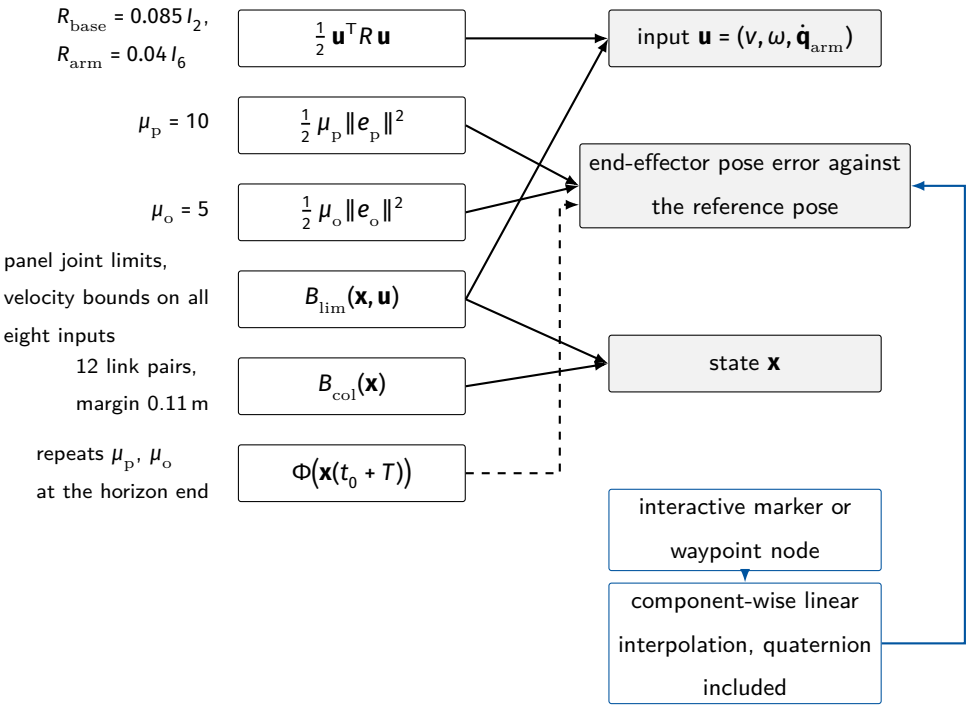


Figure 3.5: Terms of the objective in Equation (3.3), the values configured for them in Table 3.3, and the quantities each term acts on. The ratio between R_{base} and R_{arm} is what distributes a commanded end-effector motion between base and arm. In blue, the path by which the reference reaches the two end-effector terms: the reference manager interpolates between successive knots component-wise, the quaternion included, which is why the waypoint node commands one constant orientation for a whole path.

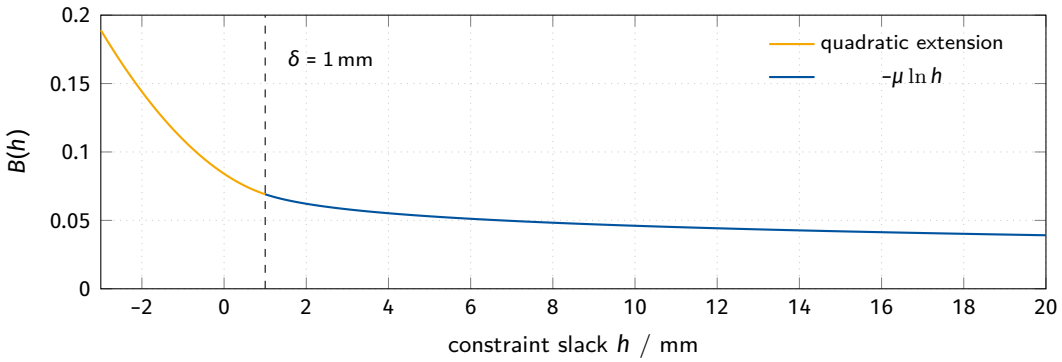


Figure 3.6: Relaxed barrier penalty of Equation (3.4) at the values configured for the self-collision term, $\mu = 10^{-2}$ and $\delta = 1\,\text{mm}$. The slack h is the amount by which a constraint is satisfied, so $h = 0$ is the constraint boundary and $h < 0$ a violation. Left of δ the logarithm is replaced by its quadratic extension, which keeps the penalty and its derivatives finite for the violated states that the real robot occasionally reports. For self-collision the slack is the distance of a link pair minus the 0.11 m margin, so the quadratic extension acts only on a pair that is closer than $0.11\,\text{m} + \delta$.

Three groups of constraints use this penalty: arm joint position limits, velocity limits on all eight inputs, and a minimum distance between selected pairs of links. The position limits are overridden in the task file instead of being read from the URDF, which bounds every UR10e joint at $\pm 360°$. Those bounds are far wider than the joint limits configured in the arm's own safety panel, so a plan that satisfies the URDF can still drive the arm into a safety limit and provoke a protective stop that OCS2 never sees. The task file therefore copies the Normal-mode panel limits of the installation used in this work (Figure 3.7): $-183°$ to $3°$ for the base and shoulder joints, $\pm 153°$ for the elbow, $-228°$ to $48°$ for wrist 1, $\pm 138°$ for wrist 2 and $\pm 183°$ for wrist 3. The velocity bounds are $\pm 0.1\,\mathrm{m/s}$ and $\pm 0.1\,\mathrm{rad/s}$ for the base and $\pm 0.3\,\mathrm{rad/s}$ per arm joint, matching the caps that the bridge enforces downstream (Section 3.4).

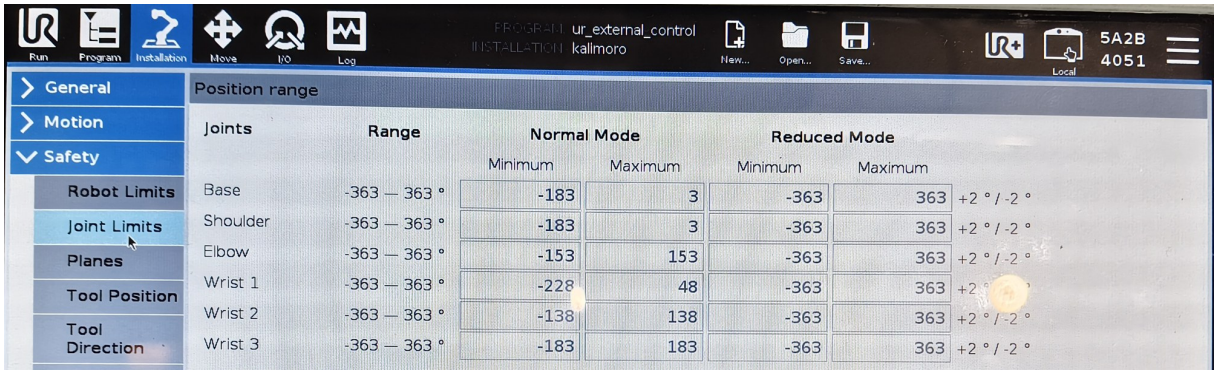
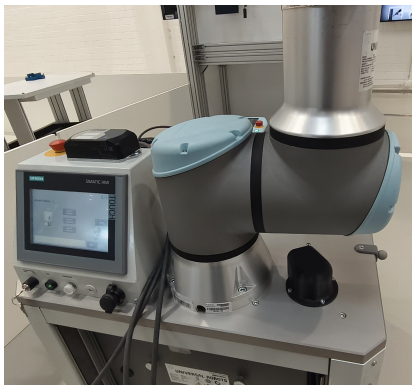


Figure 3.7: Joint-limit page of the UR10e safety configuration on the teach pendant, installation `kalimoro`. The Normal-mode ranges equal the bounds in `task_hardware.info`; the Reduced-mode ranges are not used in this work.

Self-collision is singled out here because its evaluation cost scales with the number of geometry pairs, and it was therefore the first term addressed when solve time had to be reduced. Every pair costs a distance query and its derivative with respect to the state at each node of the time grid on which the solver linearises the problem [CARP19]. That grid comes from the rollout and not from the 2 ms step at which the backward pass integrates the Riccati equation; since the rollout integrates adaptively, the number of nodes follows the solution rather than the horizon [OCS2]. It can be counted from the recordings, because a published policy carries the solver's own grid without resampling: over the 713 policies of the hardware lap of Chapter 5, one solve spans a median of ten nodes across the 1.24 s it publishes, with a quartile range of nine to ten, and the first interval equals the requested 10 ms rollout step of Table 3.3 in every one of them. Cutting the pair set from 32 to 12 therefore removes about two hundred distance queries, and as many derivative evaluations, from each solve. The constraint value of a pair is its distance minus the minimum distance [OCS2].

Two platform bodies are within reach of the arm. The chassis top is the mounting plane of the arm, and the shoulder axis lies only about 0.18 m above it; the operator panel (`hmi`), a box of 0.21 m ×

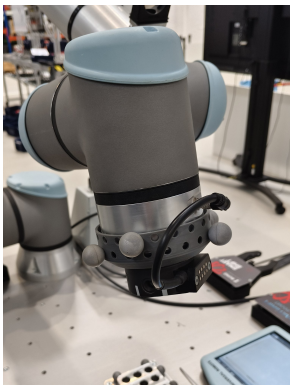
0.30 m × 0.25 m, starts 0.075 m beside the shoulder axis and covers the height of the shoulder. On the arm side, the five moving links from `upper_arm_link` to `wrist_3_link` carry attachments: six marker rings of the motion capture system, which protrude at most about 1 cm from the link surface, and six tube holders, which protrude about 3 cm to 6 cm (Figure 3.8). Pairing both platform bodies with these links and attachments gave the original set of 32 pairs. Pairs whose distance does not change with the arm configuration, such as the shoulder link against the chassis, were never part of it.



(a) Arm base on the chassis top, next to the operator panel (`hmi`).



(b) Tube holders on the upper arm and the forearm.



(c) Wrist 3 with the tool flange (`tool0`) and motion-capture markers.

Figure 3.8: Platform bodies and arm attachments that enter the self-collision model.

The reduction to twelve pairs, listed in Table 3.2, removes pairs that are redundant rather than pairs that are harmless. An attachment is dropped when the pair formed by its parent link and the same platform body already keeps it clear. For the rings this holds with any margin larger than their protrusion. For the tube holders it required raising the minimum distance from 0.08 m to 0.11 m, which leaves a clearance of about 5 cm for a dropped holder in its least favourable orientation. The two largest holders sit on the forearm, the link that most often dips towards the chassis during operation, and their pairs with the chassis are kept explicitly. The dropped pairs are listed in a comment of the task file but are not part of the model. Their effect on solve time was not measured in isolation; the reduction cut the number of distance queries per solve by 62.5 %, and the UR safety layer remains the last line of protection against a contact the reduced set does not see.

Figure 3.9 shows two arm postures in which the wrist comes close to the chassis. In both, the elbow is raised and the forearm points downwards. In the first posture the three wrist links arrive at the edge of the chassis top. In the second the forearm reaches past that edge, and the wrist links lie beside the side wall of the chassis, below the top plate. The pairs concerned are those of `chassis` with `forearm_link` and with `wrist_1_link` to `wrist_3_link`.

Table 3.2: Self-collision pairs between the arm and the two platform bodies, before (32) and after (12) the reduction. A dropped pair is covered by the pair of its parent link with the same platform body and the minimum distance of 0.11 m.

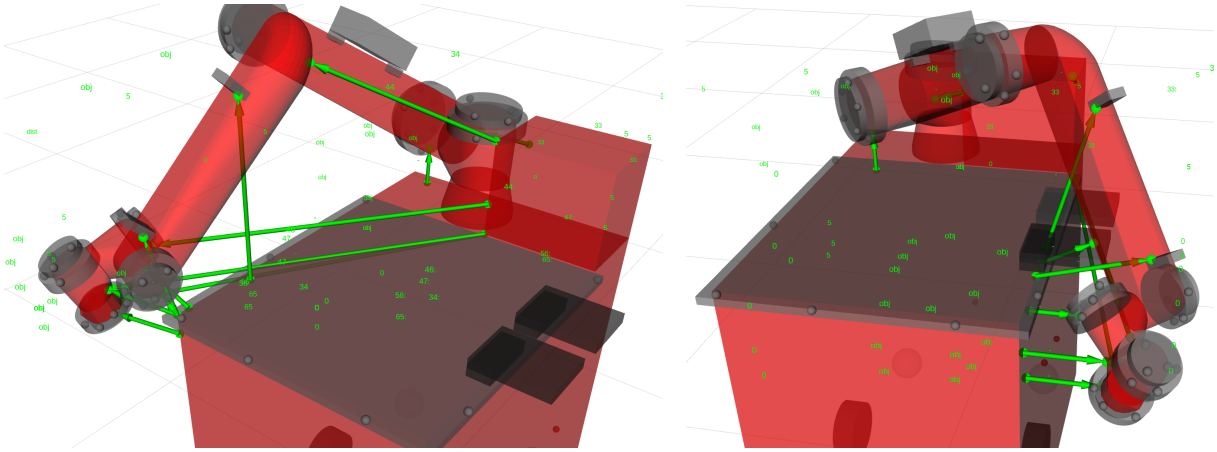
Arm-side body	Protrusion	`hmi`	`chassis`
`upper_arm`, `forearm`, `wrist_1–3`	link itself	kept (5)	kept (5)
marker rings `ring2`, `ring2b`, `ring3–ring6`	≤ 1 cm	dropped (6)	dropped (4)
tube holders `bottom_1–4`	3 cm to 6 cm	dropped (4)	dropped (4)
tube holders `top_3`, `top_4` (forearm)	3 cm to 6 cm	dropped (2)	kept (2)
Pairs before / after		17 / 5	15 / 7

The solver settings and the horizon are collected in Table 3.3. Three of them interact and are worth spelling out. Only one SLQ iteration per solve is allowed. The solver starts a new solve as soon as the previous one has finished (Section 3.4), so the previous solution, computed a few hundred milliseconds earlier over a 1.2 s horizon, is a usable initial guess; a second iteration would roughly double the solve time and delay the next fresh observation by the same amount [FARS17]. The policy that the solver publishes is declared valid for a *solution time window* of 1.0 s, shorter than the horizon; the bridge holds the robot once a policy is exhausted rather than extrapolating it (Section 3.4). Real-time operation therefore requires

$$2 \cdot \bar{t}_{\text{solve}} < T_{\text{sol}} < T, \tag{3.5}$$

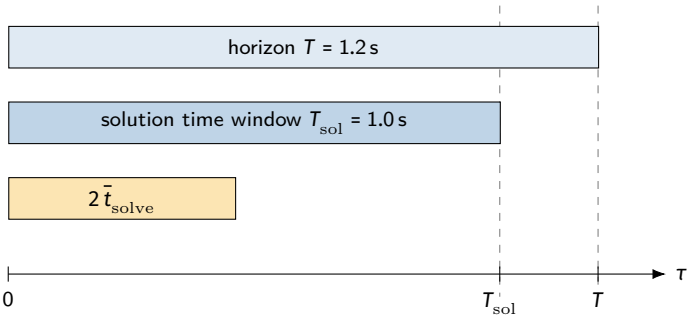
with $\bar{t}_{\text{solve}}$ the average solve time, T_{sol} the solution time window and T the horizon: a policy must arrive before the previous one expires, and it must not claim validity beyond the horizon it was computed for. This invariant is checked from the solver's diagnostics topic at the start of every session and is the quantity that governs the failure mode analysed in Section 5.2. Figure 3.10 places the three quantities on one time axis.

Derivatives are not written by hand. OCS2 traces the cost terms, the constraints and the system equation with CppAD and compiles the resulting derivative code into shared libraries on first use, which is the delay mentioned in Section 3.2 [CppAD; OCS2]. The cache is keyed by the model, not by the tuning: changing the kinematic model, the fixed joints or the collision pair list invalidates it and requires a regeneration, whereas changing a weight, a limit or the horizon takes effect on the next restart. This distinction matters in practice, because a stale cache silently keeps the old model while the configuration file suggests otherwise.



(a) Wrist links at the edge of the chassis top **(b)** Wrist links beside the side wall of the chassis

Figure 3.9: Arm postures in which the wrist comes close to the chassis, drawn with the collision bodies of the model in RViz. Red bodies are the chassis, the operator panel (`hmi`) and the arm links; marker rings and tube holders are grey. Each green arrow joins the closest points of one body pair that the distance node evaluates. The green text is the label output of that node and is not evaluated here.



If $2\,\bar{t}_{\mathrm{solve}}$ reaches past T_{sol}, a policy expires before its successor arrives and the bridge holds. If T_{sol} passed T, a policy would claim validity beyond the horizon it was computed for.

Figure 3.10: The three quantities of Equation (3.5) on one time axis, at the everyday operating point of Table 3.3. The ordering $2\,\bar{t}_{\mathrm{solve}} < T_{\mathrm{sol}} < T$ is a design condition checked before a session, not a measurement, and the length of the $2\,\bar{t}_{\mathrm{solve}}$ bar is illustrative; the solve times actually reached on the two plants are reported in Section 5.2, where the MPC loop runs at the wider Experiment A setting.

Table 3.3: OCS2 operating point for on-robot operation, as configured in `task_hardware.info`. Velocity limits appear twice by design: once as a soft constraint inside the optimisation and once as a hard clamp in the bridge.

Group	Parameter	Value
Model	state / input dimension	9 / 8
	base frame, end-effector frame	`footprint`, `tool0`
	fixed joints	both wheel joints
Solver	algorithm	SLQ
	iterations per solve	1
	threads	8
	backward-pass step, integrator	2 ms, ODE45
	rollout step	10 ms
	feedback policy	disabled (feed-forward)
MPC	horizon T	1.2 s
	solution time window T_{sol}	1.0 s
	bridge loop rate f_{MRT} (`mrtDesiredFrequency`)	50 Hz
	solve rate	$\min(f_{\mathrm{MRT}}, 1/t_{\mathrm{solve}})$
	`mpcDesiredFrequency`	10 Hz, warning threshold only
Cost	R_{base}, R_{arm}	$0.085\, I_2$, $0.04\, I_6$
	μ_{p}, μ_{o} (running and final)	10, 5
Limits	base velocity	± 0.1 m/s, ± 0.1 rad/s
	arm joint velocity	± 0.3 rad/s
	arm joint positions	UR safety-panel limits
	self-collision pairs, margin	12, 0.11 m

3.4 Hardware Interfaces and ROS Control Layer Integration

The MRT bridge is the only component written specifically for this deployment. It is the MRT client of the MPC–MRT split (Section 2.2.1) adapted to the real robot, and it closes the loop between the OCS2 policy and `ros2_control` on the IPC [ros2_control]. Its practical role is to decouple the actuation rate from the solve rate. A policy arrives only every few hundred milliseconds, but it is a function of time over the solution time window, so the bridge can evaluate it at the current time in every cycle and keep commanding the robot between solves. The bridge is also the place where a policy that has become too old is detected and replaced by a hold.

The bridge has a single clock, the loop rate f_{MRT} set by `mrtDesiredFrequency` in the task file. Each cycle publishes an observation, collects a newly arrived policy, runs the checks of Table 3.4 and evaluates the policy into a base twist and an arm setpoint. The observation also paces the solver: the MPC node solves inside the callback of each observation it receives, with a queue depth of one and no throttle of its own [OCS2]. The solve rate on the robot is therefore $\min(f_{\mathrm{MRT}}, 1/t_{\mathrm{solve}})$, with t_{solve} the duration of one solve. The `mpcDesiredFrequency` entry of 10 Hz does not throttle anything on this path; the bridge uses it only as the warning threshold in Table 3.4.

The loop rate was lowered from 100 Hz to 50 Hz after the sessions reported in Chapter 5, which ran at 100 Hz. At 100 Hz the bridge sampled the base faster than its state source changes: the drive controller publishes odometry at 50 Hz, so every second observation carried the same base pose as the one before it. That doubled the observation traffic over the network and the work of the bridge on the four-thread IPC without adding information. The solver was not affected, because with solves of a few hundred milliseconds it runs back to back at either rate. The lower rate keeps the other requirements on the loop satisfied with margin. The base twist is held for one cycle, which at the 0.1 m/s bound is a step of 2 mm instead of 1 mm. The 0.2 s staleness window is still checked ten times, and the 0.5 s command timeout of the drive controller spans 25 cycles. The commanded arm speed is bounded by the step clamp described below, independently of the loop rate. The price is that a new policy may wait up to one cycle, 20 ms, before it is first used.

Each cycle begins with an observation. The base pose (p_x, p_y, ψ) is taken from the drive controller's odometry, with the yaw extracted from the reported quaternion, and $\mathbf{q}_{\mathrm{arm}}$ is looked up by joint name in `/joint_states`; a missing joint aborts the cycle with a throttled warning instead of silently inserting a zero. The observation timestamp is the later of the two message stamps, expressed relative to the origin fixed at the first accepted measurement, which is the relative time axis that makes the deployment insensitive to clock offsets.

Before the first command, the bridge performs a reset handshake with the MPC node. The initial reference must not move the robot, so the bridge computes the current end-effector pose by forward kinematics from the latest observation and sends it as a two-knot target trajectory inside the reset request: the solver starts by holding the pose the robot is already in. The bridge guards the handshake against both ways it can fail. It waits for the reset service rather than assuming it exists, logs an actionable error after 15 s and keeps waiting, and retries the request if the response is lost—a single dropped reply over the network must not leave the bridge dead. After a handshake the time-axis baseline is reset as well, because the blocking call may have consumed several seconds and the next observation would otherwise look like a time jump. Figure 3.11 shows the exchange and the two ways it can fail.

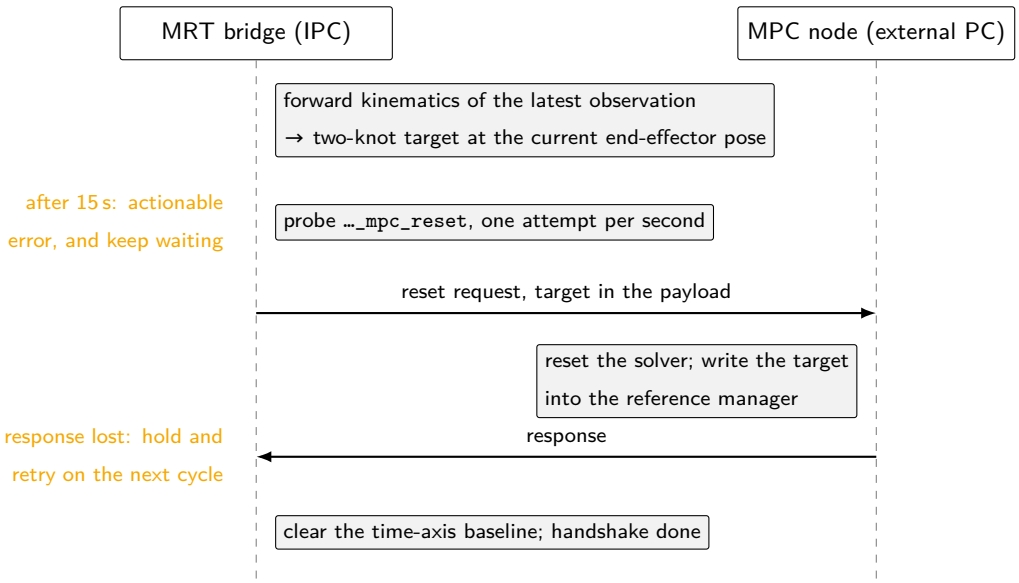


Figure 3.11: Reset handshake that precedes the first command. The bridge derives the initial reference from its own latest observation, so the solver starts by holding the pose the robot is already in. Both guards, in orange, sit in the bridge: it waits for the service rather than assuming it exists, and a lost response leaves it holding and retrying instead of dead. Clearing the time-axis baseline afterwards is necessary because the blocking call may have consumed several seconds, which the next observation would otherwise present as a time jump.

The base command is the first two elements of the optimised input, published as a stamped twist on `/cmd_vel_ocs2` rather than directly on the drive controller's command topic. A `twist_mux` arbitrates between sources by priority, with manual teleoperation above OCS2, so a higher-priority teleoperation source preempts or stops a 400 kg platform that is executing an optimised trajectory [twist_mux]. The published twist is clamped to the same base velocity bounds that the optimisation uses as a soft constraint; the clamp is the hard backstop for the case where the soft constraint is violated.

The arm command needs additional handling, because the joint-trajectory controller accepts a position setpoint over its topic interface without enforcing any velocity limit [ros2_control]. The bridge samples the optimised arm state 0.6 s ahead and publishes it as a single trajectory point with that time from start, but it first clamps each joint's step to $\dot{q}_{\max} \cdot t_{\text{preview}} = 0.18$ rad around the currently measured position. Because the next cycle repeats the computation 20 ms later, the arm still advances towards the target, at a rate bounded by 0.3 rad/s. When the clamp engages, the bridge warns; frequent warnings indicate that the optimised state is jumping between solves, which later proved useful as a diagnostic and is taken up in Section 5.2.

Whenever the bridge cannot produce a trustworthy command it publishes a hold: a zero twist and an arm trajectory pinned at the measured joint positions. Getting this right required two details that are invisible in the algorithm but decide whether the robot actually stops. The twist is stamped at the moment of publication, because a hold issued right after a blocking call would otherwise carry a stale stamp and be rejected by the drive controller as timed out. The arm trajectory carries a zero header stamp, which the controller reads as "start now"; a wall-clock stamp arrives a few milliseconds late and the whole trajectory is rejected as ending in the past, so the hold never applies. Table 3.4 lists the conditions that trigger a hold, and Figure 3.12 places them in the order the cycle evaluates them.

Below the bridge, the control chain is the production HERo stack with two deliberate deviations. The UR driver is started in headless mode with the joint-trajectory controller active, so that whole-body control does not depend on an operator starting a program on the teach pendant; the trade-off is that the pendant must be in remote mode when the driver starts, otherwise the external control program has to be re-sent. The controller stopper, which deactivates arm controllers when no program runs on the arm, keeps both the joint-trajectory controller and the drive controller in its list of controllers to leave alone. This preserves the safety interlock of the arm while ensuring that a protective stop does not silently drop the controller through which OCS2 actuates.

Two limitations of the integration follow from the choices above rather than from tuning, and both bound what the experiments in the following chapters can show. The controller's notion of where the robot is comes from wheel odometry alone, so end-effector poses reconstructed from the internal state are consistent within a run but not absolute; and the optimisation reasons about the robot's own geometry only, so a whole-body plan is collision-free with respect to the platform and blind to the workspace around it.

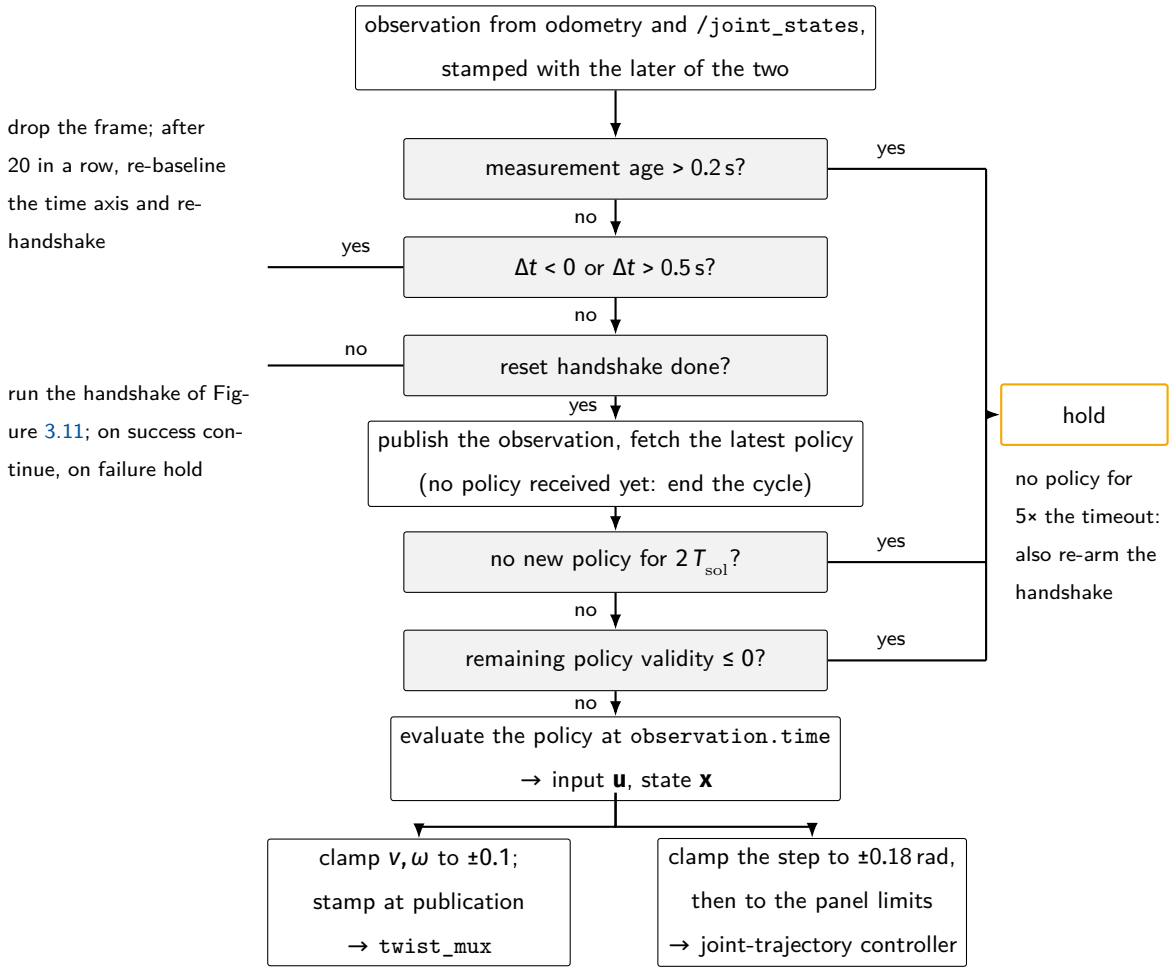


Figure 3.12: One cycle of the MRT bridge, which runs at 50 Hz on the IPC. The guards are evaluated in the order shown; each either passes the cycle on or ends it in the hold, which publishes a zero twist stamped at publication together with an arm trajectory pinned at the measured positions under a zero header stamp. The arm step limit of 0.18 rad is the product of the arm velocity bound and the preview time, 0.3 rad/s × 0.6 s, so the arm still advances towards the optimised state over successive cycles. Thresholds and their origin are listed in Table 3.4.

Table 3.4: Guards implemented in the MRT bridge. Most thresholds are launch arguments of the bridge; the policy timeout is derived from T_{sol}, and the warning threshold from `mpcDesiredFrequency`, both in the task configuration file.

Condition	Reaction
Measurement older than 0.2 s	hold; the state estimate is not trusted
Observation time steps backwards, or forwards by more than 0.5 s	drop the frame to protect the solver's time axis; after 20 consecutive drops, accept the new time as baseline and re-handshake
Remaining policy validity below 0.1 s (1/`mpcDesiredFrequency`)	warn: the robot is outrunning the optimiser
Policy validity exhausted	hold instead of evaluating a policy beyond its window
No new policy for $2\,T_{\mathrm{sol}}$, at least 1 s	hold; the external PC or the link is assumed busy
No new policy for 5× that time, at least 5 s	re-run the reset handshake, on the assumption that the MPC node restarted and no longer accepts the current observations
Commanded arm step beyond the rate limit	clamp the step and warn

4

Simulation-Based Validation

The hardware loop of Chapter 3 mixes three contributions to the recorded end-effector error: the residual of the optimiser on its own kinematic model, the lag and clamping of the MRT bridge, and the odometry-plus-forward-kinematics estimate of where the robot is. A kinematic closed loop that replaces the bridge with an ideal rollout removes the last two, so a fault in the reference, the time axis or the cost function can be seen before it is blamed on the plant. That loop is the dummy MRT of OCS2 [OCS2]. Section 4.1 states what it can show and defines the metrics used here and in Chapter 5. Section 4.2 describes the closed end-effector path that the dummy exercises, the checks that a session must pass before the same files are taken to the robot, and the residual measured on one accepted lap.

4.1 Validation Methodology and Evaluation Metrics

The dummy MRT is not a second controller. It is the production MPC node and the production target publisher, closed through `MRT_ROS_Dummy_Loop` instead of the hardware bridge (Section 3.4). The dummy integrates the latest policy with the same kinematic rollout the solver uses and feeds the resulting state back as the next observation [OCS2]. The task file configures it for 100 Hz, a rate the recorded session does not reach (Section 4.2.4), which costs nothing here because the loop has no plant to keep up with. There are no actuator dynamics and no wheel-odometry drift, and the loop is not bound to a wall clock: it advances its own time axis one step per iteration, so a policy is never older, on that axis, than the solve that produced it. The end-effector error that remains is the optimiser's residual against its own model: input costs and velocity limits prevent a perfect match to the interpolated reference, and a finite horizon cannot see the whole path [RAWL17; MAYN00].

That residual is diagnostic. If the dummy already leaves the commanded path, the later hardware error is not an execution problem. If the dummy tracks and the hardware does not, the gap is in the bridge, the actuators, or the state estimate, which is the comparison drawn in Chapter 5. The dummy does not address real-time behaviour on the IPC: solve time, policy exhaustion and the

observation-age guard never appear, because the observation is produced in the same process tree as the solver.

A second simulated-hardware mode exists in the HERo stack: mock `ros2_control` interfaces that echo commands into states while still running the MRT bridge. That mode keeps the timing and clamping of Section 3.4 and is used as a stepping stone onto the physical platform [ros2_control]. It is not the subject of this chapter.

4.1.1 Internal metrology

Desired and attained end-effector poses are both computed inside the waypoint node from the same MPC observation. The desired pose is the linear interpolation of the commanded knots at `observation.time`, which is the interpolation OCS2 itself uses for the position part of the reference. The attained pose is Pinocchio forward kinematics of that observation, with `tool0` as the end-effector frame (Section 3.1) [CARP19]. The error signal therefore carries no cross-topic time-alignment residual. The node publishes it on `/ee_waypoint_target/ee_metrics`, at 20 Hz in the hardware sessions of Chapter 5 and at ten samples per second of OCS2 time in the dummy session of Section 4.2.4, where the loop produces observations more slowly.

On the dummy this is a consistent kinematic measurement. On hardware the same pipeline reconstructs the end-effector from wheel odometry and joint encoders. Drift of the base pose then contaminates every later metric. The figures and tables of this chapter, and the internal traces of Chapter 5, are therefore internal metrology: they score consistency between the commanded and attained poses that the controller itself reconstructs, not pose error relative to the hall.

4.1.2 Time-aligned pose error

Let $\mathbf{p}_{\mathrm{d}}(t)$ and $\mathbf{p}_{\mathrm{a}}(t)$ be the desired and attained positions in the `world` frame, and $q_{\mathrm{d}}(t)$, $q_{\mathrm{a}}(t)$ the corresponding unit quaternions. The position and orientation errors used throughout are

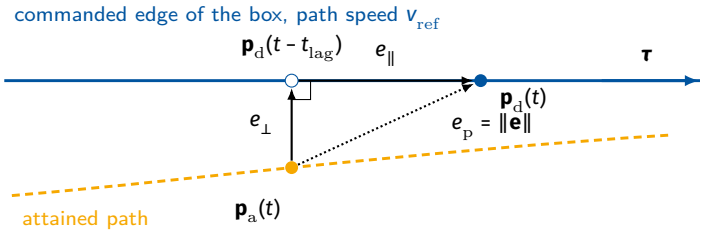
$$e_{\mathrm{p}}(t) = \|\mathbf{p}_{\mathrm{d}}(t) - \mathbf{p}_{\mathrm{a}}(t)\|, \qquad e_{\mathrm{o}}(t) = 2\,\mathrm{atan2}\big(\|\mathbf{v}\|, |w|\big), \tag{4.1}$$

where $(w, \mathbf{v})$ are the scalar and vector parts of $q_{\mathrm{d}}\, q_{\mathrm{a}}^{-1}$ and e_{o} is folded into $[0, \pi]$ so that the quaternion double cover does not appear as a 2π jump [LYNC17]. On a measured edge of the path the position error is split into an along-track component $e_{\parallel} = \mathbf{e} \cdot \boldsymbol{\tau}$ and a cross-track remainder $e_{\perp} = \|\mathbf{e} - e_{\parallel}\,\boldsymbol{\tau}\|$, with $\boldsymbol{\tau}$ the unit tangent of that edge. The ratio

$$t_{\mathrm{lag}} = e_{\parallel} / v_{\mathrm{ref}} \tag{4.2}$$

converts the along-track error into an equivalent time, with v_{ref} the commanded path speed. On hardware, t_{lag} is the quantity that can be compared with the policy-lag diagnostic of the MRT

bridge (Section 5.2); the dummy has no such lag, so a large t_{lag} there would mean the optimiser is chronically behind its own reference. Figure 4.1 shows the decomposition on one measured edge.



$e_{\parallel} > 0$: the end-effector trails the reference along the edge, by the time $t_{\mathrm{lag}} = e_{\parallel}/v_{\mathrm{ref}}$ of Equation (4.2); the hollow marker is where the reference was t_{lag} earlier. $e_{\perp}$: the part of the error that lies off the commanded edge, which no lag accounts for.

Figure 4.1: Decomposition of the time-aligned end-effector position error on one measured edge. Desired and attained positions are taken from the same MPC observation (Section 4.1.1), so both points belong to the same instant t. The error splits along the unit tangent $\boldsymbol{\tau}$ of the edge into an along-track component $e_{\parallel}$, which Equation (4.2) converts into the equivalent time t_{lag}, and a cross-track remainder $e_{\perp}$. The split matters on hardware because t_{lag} can be compared with the measured policy lag of the MRT bridge (Section 5.2) while $e_{\perp}$ cannot.

These samples are only admitted on the measured path. The waypoint node tags every metric with a segment identifier: `segment_id` ≥ 1 is an edge of the closed box; 0 is the approach from the current end-effector to the entry point; −10, −11 and −12 are the lead-in, lead-out and optional park legs of ISO 9283 cl. 8.2 [ISO 9283]. Post-processing keeps `segment_id` ≥ 1. The transients of starting, stopping and parking do not enter e_{p}, t_{lag} or the path metrics below.

4.1.3 Path metrics: Hausdorff distance and ISO path accuracy

Time-aligned error assumes that the commanded and attained poses share a clock and a speed profile. That is true for the OCS2 loop, where the reference is a time-parameterised `TargetTrajectories` message. The assumption has a cost: an end-effector that follows the commanded edge exactly but late is charged the full along-track distance $e_{\parallel}$, so e_{p} alone cannot say whether the attained path has the commanded shape. A second metric therefore treats the command as a geometric path, not as a trajectory. The (symmetric) Hausdorff distance between the commanded point set A and the attained point set B is [HUTT93]

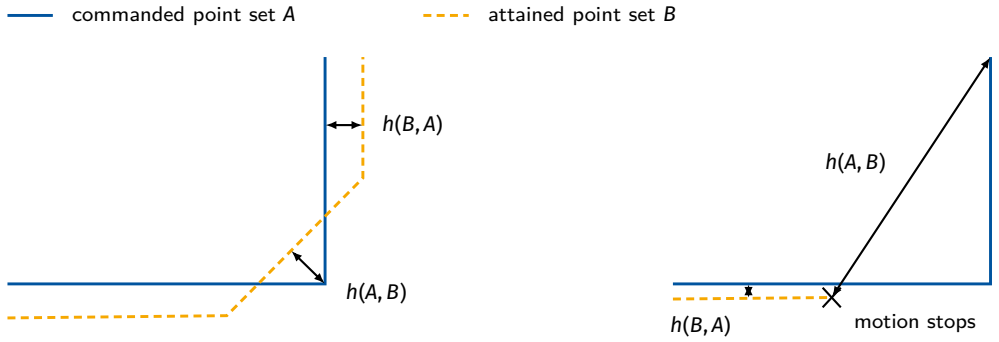
$$h(A,B) = \max_{a \in A} \min_{b \in B} \|a - b\|, \qquad H(A,B) = \max\big(h(A,B),\, h(B,A)\big). \tag{4.3}$$

The directed distance $h(B,A)$ is small whenever every attained sample lies near the command, including the degenerate case of a robot that stops halfway; $h(A,B)$ penalises shortcuts and unfinished laps. Both directions are required, as Figure 4.2 shows. The classical $\max$ is sensitive to a single

spike, so results are also reported with a mean nearest-neighbour form in the spirit of the modified Hausdorff distance [DUBU94],

$$\overline{H}(A,B) = \frac{1}{2|A|}\sum_{a\in A}\min_{b\in B}\|a-b\| + \frac{1}{2|B|}\sum_{b\in B}\min_{a\in A}\|a-b\| \tag{4.4}$$

together with the 95th percentile of the nearest-neighbour distances and the raw $H(A,B)$. The command path is resampled at a spacing well below the expected error (one millimetre is enough for the box of Section 4.2) before the nearest-neighbour search, otherwise four or eight corners would inflate every distance.



(a) both laps complete: the two directed distances are of comparable size

(b) the attained path stops halfway: only $h(A,B)$ exposes it

Figure 4.2: Why Equation (4.3) needs both directed distances. In (a) the attained path cuts the corner and bulges outside the command, and the two directions report deviations of similar size. In (b) the robot covers the first part of the path and stops: every attained sample still lies close to the command, so $h(B,A)$ stays small, while $h(A,B)$ grows to the distance from the unvisited far end of A to the nearest attained sample. Reporting $h(B,A)$ alone would rank the unfinished lap in (b) above the complete one in (a).

ISO 9283 defines path accuracy AT_p along a commanded path as the maximum, over the path, of the distance between that command and the barycentre of n attained cycles [ISO 9283]. With $n = 1$, AT_p reduces to the maximum deviation of a single attained path from the command, which is close in spirit to $h(B,A)$ but still parameterised by the command, not by an unordered cloud. Pose accuracy AP_p (cl. 7.2.1) is the distance from each commanded corner to the barycentre of the attained poses at that corner [ISO 9283]. Cycle counts and the three mandatory speed levels of ISO Table 3 are not exercised in this chapter [ISO 9283]. One slow lap on the dummy is a pipeline check, not an ISO test.

Task completion time, the wall-clock duration of the measured lap, is recorded alongside the geometric scores. Hausdorff distance ignores speed; a slow but accurate motion would otherwise look like a win.

Table 4.1 summarises which quantity answers which question. Dummy sessions use the first two rows as live diagnostics in RViz and in the metric topic. Hausdorff and AT_p are computed offline from the same bag export that Chapter 5 uses.

Table 4.1: Metrics used in the closed-path experiments. Dummy sessions produce the first two rows as a planner residual. Hausdorff distance and AT_p are evaluated offline on samples with `segment_id` ≥ 1. Clause labels for AT_p and AP_p follow ISO 9283 [ISO 9283].

Quantity	Question	Notes
e_{p}, e_{o}	How far is the end-effector from the command at this instant?	Equation (4.1); same observation for both poses
$e_{\parallel}$, $e_{\perp}$, t_{lag}	Is the error along-track lag or cross-track deviation?	measured edges only; Equation (4.2)
H, $\overline{H}$, P_{95}	Do the two paths have the same shape?	time-free; both directions required
AT_p	ISO path accuracy of the whole-body loop	cl. 8.2; here one cycle, $n = 1$
AP_p	Did the end-effector reach the corners?	cl. 7.2.1; defined here, not reported
T	How long did the measured lap take?	wall clock; Hausdorff does not capture this

4.2 Kinematic Closed-Loop Simulation (Dummy MRT)

The dummy launch starts, on one machine, the URDF exporter, the MPC node, the dummy MRT, the robot-state publisher, RViz, and the waypoint target node. The interactive-marker target used in the OCS2 demo is switched off, so the waypoint node is the only publisher on the MPC target topic. The task file is `task_experiment_a.info`, the same file the hardware host launch loads, and the waypoint file is `experiment_a_waypoints.yaml`. What is being checked is the configuration that will run on the robot, not a separate simulation tune.

Figure 4.3 shows the resulting graph. Compared with Figure 3.4 there is no Zenoh boundary and no `ros2_control` layer. The dummy loop calls `rolloutPolicy` on the kinematic model, advances `observation.time` by 10 ms, and publishes the new observation. The waypoint node consumes that observation both to build knot times and to evaluate Equation (4.1).

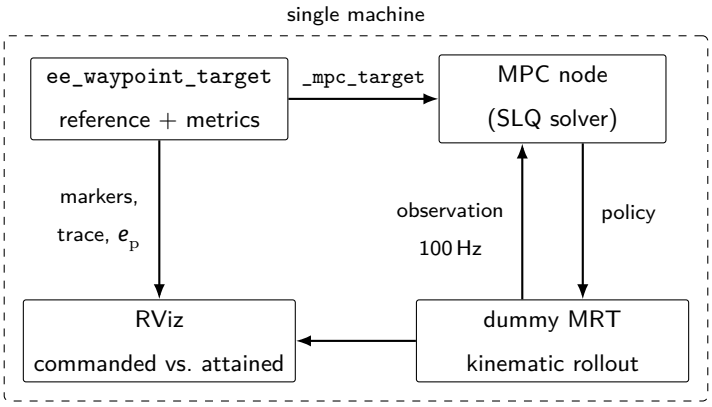


Figure 4.3: Dummy-MRT closed loop used to accept the Experiment A configuration. The hardware bridge of Figure 3.4 is replaced by a kinematic rollout of the latest policy; the waypoint node is otherwise unchanged.

4.2.1 Commanded path

The commanded path is an eight-corner rectangular parallelepiped whose edges are aligned with the world axes after the box is landed (Figure 4.4 and Table 4.2). ISO 9283 cl. 6.8.1 replaces a cube by a parallelepiped when the travel along one axis is much smaller than along the others [ISO 9283]; here z is an arm coordinate and x, y can recruit the base. The implemented box is smaller than the workspace-filling $2.0\,\mathrm{m} \times 2.0\,\mathrm{m} \times 0.40\,\mathrm{m}$ parallelepiped discussed as a design target. Its horizontal extents are 0.60 m by 0.80 m and its height is 0.25 m, with the lower face at 1.30 m and the upper face at 1.55 m. The perimeter is 4.50 m.

Horizontal offsets are stored relative to the end-effector pose at trigger time (`origin_frame: ee`); the z values are absolute world heights. Anchoring on the current `tool0` keeps the first corner next to the standby pose, so the approach gate is a short motion rather than a reach across the cell; the first offset is only 0.41 m. The far corner lies 1.35 m from that same end-effector, which is enough to pull the base on the long edges without demanding a $2\,\mathrm{m}$ translation on every lap. The standby joint vector written in `task_experiment_a.info` (dummy only; the hardware bridge ignores it) is the MoveIt named state `bent`: shoulder pan, lift, elbow and wrist 1 at $-90°$, wrist 2 at $90°$, wrist 3 at zero [MoveIt 2].

The visit order is a Hamiltonian cycle on the eight corners in which every consecutive pair differs in a single coordinate:

$$\mathrm{P1} \xrightarrow{x} \mathrm{P2} \xrightarrow{y} \mathrm{P3} \xrightarrow{x} \mathrm{P4} \xrightarrow{z} \mathrm{P5} \xrightarrow{x} \mathrm{P6} \xrightarrow{y} \mathrm{P7} \xrightarrow{x} \mathrm{P8} \xrightarrow{z} \mathrm{P1}.$$

Six edges are horizontal and two are vertical. The vertical pair is a pure arm motion in z; the base cannot contribute to it. Closing the loop on a downward z edge has a consequence for the ISO lead-out, taken up below.

All eight knots share one orientation, $\mathrm{RPY} = (\pi, 0, -\pi/2)$, which is the `tool0` pose of `bent` given the $-90°$ mount of the arm (Section 3.1). A constant quaternion is required because `TargetTrajectories::getDesiredState` interpolates quaternion coefficients component-wise, not by spherical interpolation. A time-varying reference orientation would inject interpolation error into e_{o} that is not the controller's.

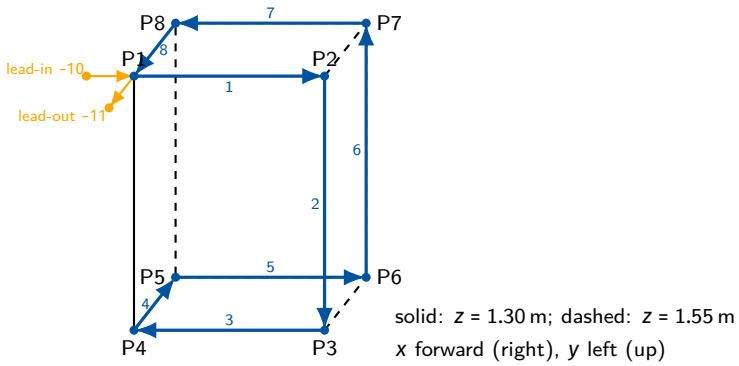


Figure 4.4: Commanded parallelepiped in the end-effector-anchored frame of `experiment_a_waypoints.yaml`, carrying the segment identifiers that the waypoint node publishes with every metric (Figure 4.5). Blue arrows are the measured Hamiltonian cycle, tagged 1 to 8 in the order they are traversed; these are the samples that post-processing keeps. The orange stubs are the ISO 9283 cl. 8.2 lead legs [ISO 9283], tagged −10 and −11: the lead-in runs 0.15 m along the backward extension of P1–P2, and the lead-out continues 0.15 m down from P1 along the closing z edge, to $z = 1.15$ m. The approach from the standby pose to the lead-in point is tagged 0 and is not part of the geometry shown.

4.2.2 Time axis, state machine, and ISO legs

Knot times are built on the OCS2 relative axis `observation.time`, never on the wall clock. A knot stamped in wall time falls outside the receding horizon and the robot either freezes or jumps to the last sample; that failure is cheap to see on the dummy and expensive on hardware. At a trigger time t_0 the node walks the landed corners, adding $\|\Delta\mathbf{p}\|/v_{\mathrm{ref}}$ on every edge. The configured path speed is $v_{\mathrm{ref}} = 0.05\,\mathrm{m/s}$, half of the $0.10\,\mathrm{m/s}$ base limit in `task_experiment_a.info`, so a

Table 4.2: Waypoint offsets of the Experiment A box, copied from `experiment_a_waypoints.yaml`. Δx, Δy are applied in the base yaw at trigger time about the current end-effector; z is an absolute world height.

Corner	Δx / m	Δy / m	z / m	To next
P1	0.40	−0.10	1.30	$+x$, 0.60 m
P2	1.00	−0.10	1.30	$-y$, 0.80 m
P3	1.00	−0.90	1.30	$-x$, 0.60 m
P4	0.40	−0.90	1.30	$+z$, 0.25 m
P5	0.40	−0.90	1.55	$+x$, 0.60 m
P6	1.00	−0.90	1.55	$+y$, 0.80 m
P7	1.00	−0.10	1.55	$-x$, 0.60 m
P8	0.40	−0.10	1.55	$-z$, 0.25 m

dummy that cannot keep up is a cost or constraint problem rather than a saturated plant. With `corner_dwell_sec` = 2.0 s the reference pauses at each corner; the 4.50 m perimeter then takes 90 s of motion, and the seven dwells inside the measured lap add 14 s, before lead-in and lead-out. A dwell of zero would be the ISO path-accuracy mode (constant speed through corners). The value used here is a pose-accuracy setting: the end-effector is given time to settle so that a later AP_p is not dominated by a corner cut [ISO 9283].

After `/start` the node does not immediately run the cycle. It lands the box once, holds the reference at the lead-in point, and waits until the measured end-effector stays within 0.02 m of that point for 0.3 s (`Approaching`). Only then does the running trajectory begin, so the first measured edge does not contain the motion from standby. The phase sequence is $\mathrm{Init} \rightarrow \mathrm{Hold} \rightarrow \mathrm{Approaching} \rightarrow \mathrm{Running} \rightarrow \mathrm{Finished}$. `auto_start` is false: the box is drawn in RViz first, and a service call starts the motion after a visual check. Figure 4.5 shows the phases, the reference each one publishes and the segment identifier it tags.

ISO 9283 cl. 8.2 requires the programmed start and end to lie outside the test path [ISO 9283]. A lead-in of 0.15 m is placed on the backward extension of P1–P2, and a lead-out of 0.15 m on the tangent of the closing edge. With the present Hamiltonian order that closing edge is the downward z leg, so the lead-out ends at z = 1.15 m. The dummy RViz configuration draws those legs in orange for exactly this reason: a lead-out that would enter the chassis is visible before anyone calls start. Returning to the standby pose after the lead-out is switched off for this path (`return_to_home: false`).

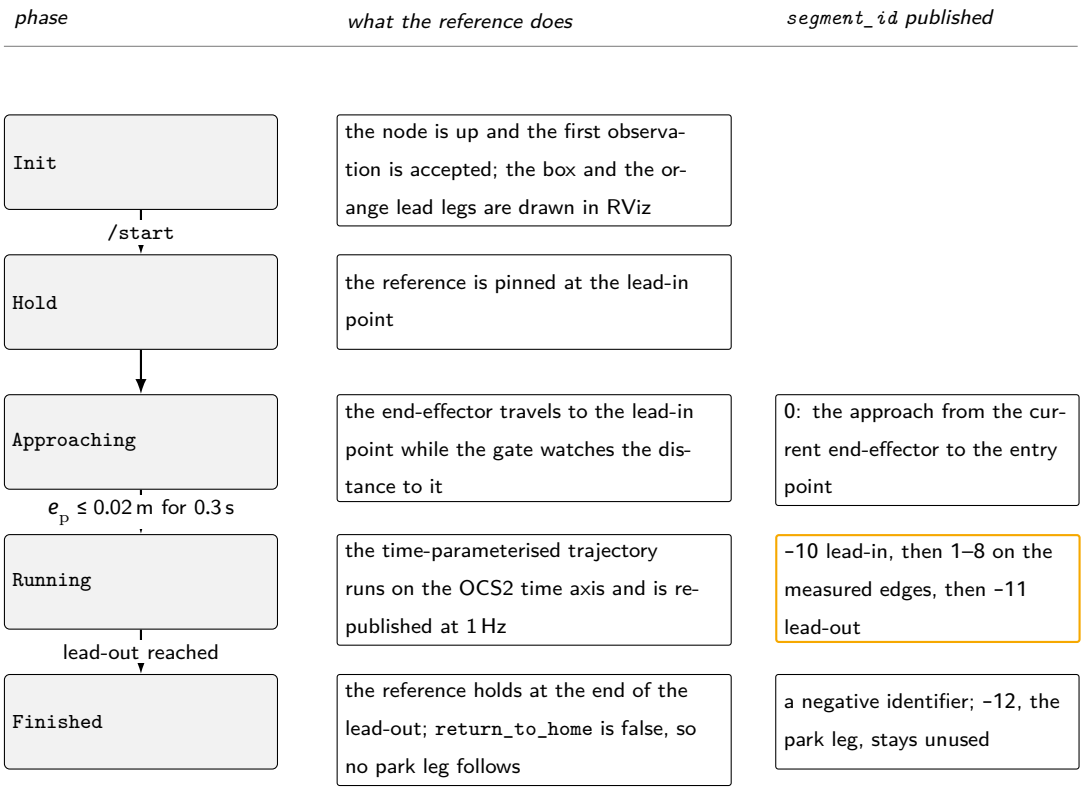


Figure 4.5: Phase sequence of the waypoint node, with the reference it publishes in each phase and the segment identifier that tags the metrics. `auto_start` is false, so `Init` waits for the operator to call `/start` after checking the landed box in RViz. The gate between `Approaching` and `Running` keeps the motion from standby out of the first measured edge; it tests position only, so a residual orientation error at entry is admitted into that edge (Section 5.1). Only the identifiers in the highlighted row survive post-processing: the metrics of Table 4.1 are computed on `segment_id` ≥ 1, which is why the approach and the two lead legs carry identifiers of their own.

The trajectory is re-published at 1 Hz. Re-sending the same knots on the OCS2 time axis is idempotent, but it restores the reference if a reset handshake of the MRT bridge has overwritten it with a two-sample hold (Section 3.4). The dummy rarely handshakes again in mid-run; the timer is still part of the node that hardware will use.

Relative to `task_hardware.info` (Table 3.3), the experiment file lowers the pre-scale diagonal of R_{base} from 8.5 to 5.5, replaces the uniform arm diagonal of 4.0 by per-joint values between 2.5 and 4.0, and raises the end-effector position weight from 10 to $\mu_p = 16$ while lowering the orientation weight from 5 to $\mu_o = 3$. A cheaper base and a tighter position term are what push the solver to reach the far corners by driving the platform instead of stretching the arm towards its reach limit. The file also lengthens the horizon from 1.2 s to $T = 1.3\,s$ and the solution time window from 1.0 s to $T_{\text{sol}} = 1.2\,s$. Solve and loop rates stay at 10 Hz and 100 Hz; the loop rate is twice the 50 Hz that

`task_hardware.info` now uses on the robot (Section 3.4). The dummy therefore validates that operating point, not the everyday hardware tune.

4.2.3 Procedure and acceptance

A dummy session is started with `experiment_a_dummy.launch.py`. After the first observation the box appears in RViz, including the orange lead legs. The operator confirms that every corner is in the cell and that the lead-out at $z = 1.15$ m clears the chassis, then calls `/ee_waypoint_target/start`. The dummy plant must reach the entry point, traverse all eight measured edges, and finish on the lead-out hold without the reference jumping to the last knot. The metric topic must show `segment_id` passing through 1–8 and returning to a negative identifier. Those checks accept the geometry, the OCS2 time axis, the data pipeline and the visualisation. They do not produce a tracking-accuracy number for the robot. Figure 4.6 shows the view in which those checks are made, before the start call, during the lap and after it.

The same YAML and task file are the input of the mock-hardware and physical runs in Chapter 5. A planar four-corner variant exists as a reduced path at constant z and shares this node; it is not used in the dummy checks reported here.

4.2.4 Residual on the accepted lap

A session that passes those checks also leaves a trace, and that trace is the optimiser's residual on its own model. The lap reported here loaded the archived Experiment A task and waypoint files, the same pair the two hardware laps of Chapter 5 used, so the three plants are scored on identical inputs. It completed all eight measured edges and held the lead-out, so it is an accepted session in the sense above. Table 4.3 lists its scores and Figure 4.7 shows the error over the measured path.

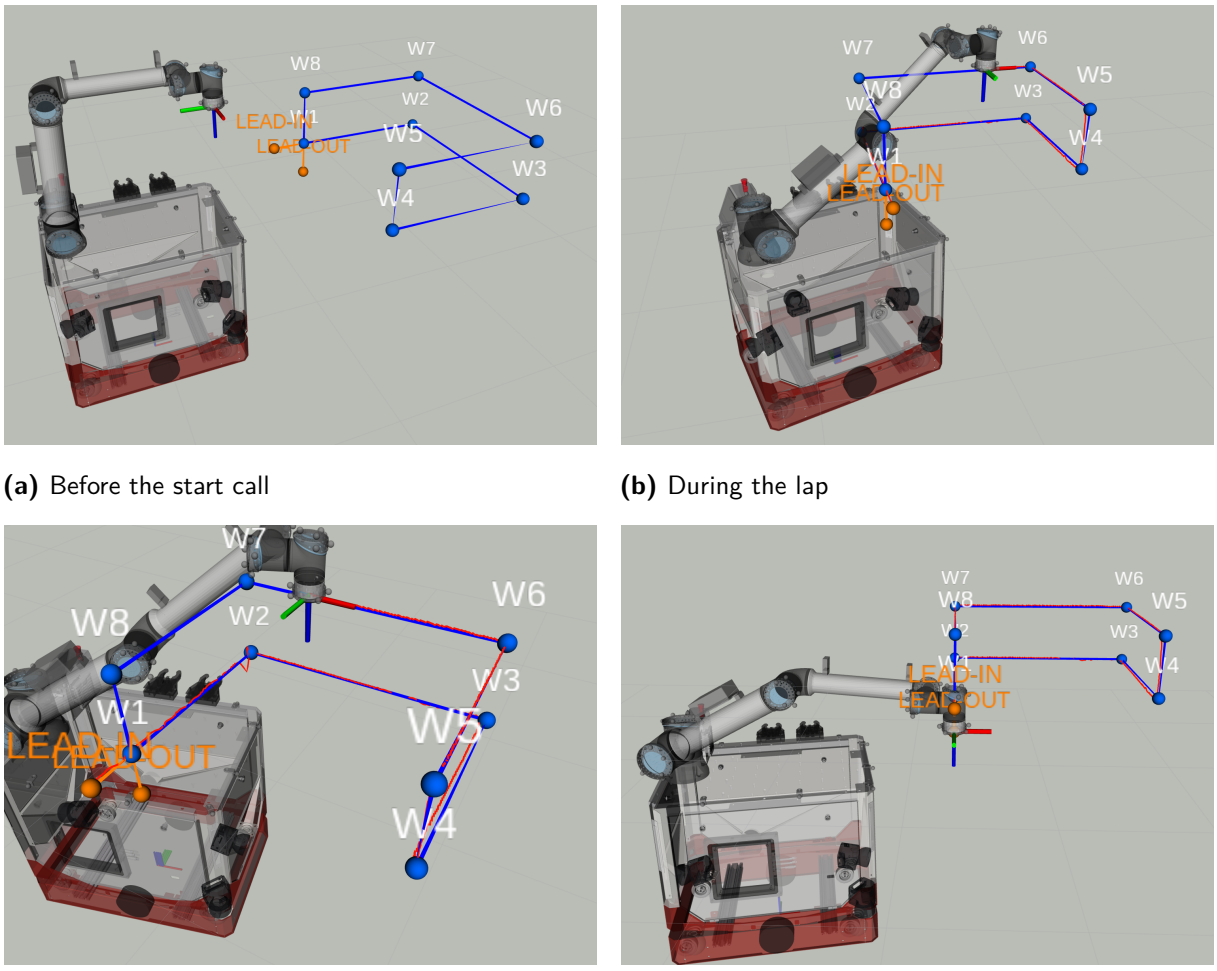
Two readings follow. The first is a lower bound: an end-effector error of about one centimetre is what the configured cost and the velocity limits leave on a model the solver itself integrates, so no part of the error measured on hardware below that level can be attributed to the plant. Per edge the RMS stays between 3 mm and 17 mm. It is consistently larger on the lower face, edges 1 to 4 at 8 mm to 17 mm, than on the upper face, edges 5 to 8 at 3 mm to 8 mm, and the single 43 mm peak sits at the end of the long $-y$ edge; the lap reported here does not separate the geometry from the tuning well enough to say which of the two causes that split. The second concerns the along-track lag. At $t_{\mathrm{lag}} = 0.10$ s the dummy is barely behind its reference, but this number does not transfer: the loop advanced 103.9 s of OCS2 time in about 244 s of wall time and produced ten observations per second of OCS2 time against the hundred the hardware loop produces, so the solver had more wall

Table 4.3: Residual of the dummy loop on the accepted Experiment A lap (`segment_id` ≥ 1), computed with the metrics of Table 4.1. Hausdorff quantities use the 1 mm resampling of the commanded polyline defined in Section 4.1.3. The same quantities for the two hardware plants are in Table 5.1.

Quantity	Dummy loop
e_{p} RMS (max)	11 mm (43 mm)
e_{o} RMS (max)	0.033 rad (0.10 rad)
H / $\overline{H}$ / P_{95}	17 mm / 5 mm / 13 mm
mean t_{lag}	0.10 s
mean solve time (median, max)	150 ms (152 ms, 276 ms)
measured lap on the OCS2 axis	103.9 s

time per simulated second than any deployed loop would grant it. The dummy therefore bounds the optimiser's residual; it does not predict the lag that Section 5.2 measures.

What the dummy still cannot close is the rest of the list in Section 4.1: actuator dynamics, odometry drift, $n \geq 3$ cycles, or a second pipeline. Those belong to the hardware chapter, using the metrics of Table 4.1 on the measured segments of the path in Figure 4.4.



(a) Before the start call

(b) During the lap

(c) During the lap, close-up of the trace

(d) After the completed lap

Figure 4.6: RViz view of the dummy loop. Blue lines and spheres are the commanded path and its corners, which the waypoint node labels W1 to W8 in the order P1 to P8 of Table 4.2. The orange legs are the lead-in and the lead-out. The red line is the attained end-effector trace, and the axis triads mark the commanded and the attained end-effector pose. In (a) the box is landed and the node waits for the start call; in (b) and (c) the trace follows the commanded edges; in (d) the trace has closed the loop and the end-effector rests on the lead-out. The screenshots come from a separate dummy session on 13 October 2026, not from the lap of Section 4.2.4. They illustrate the view and carry no score.

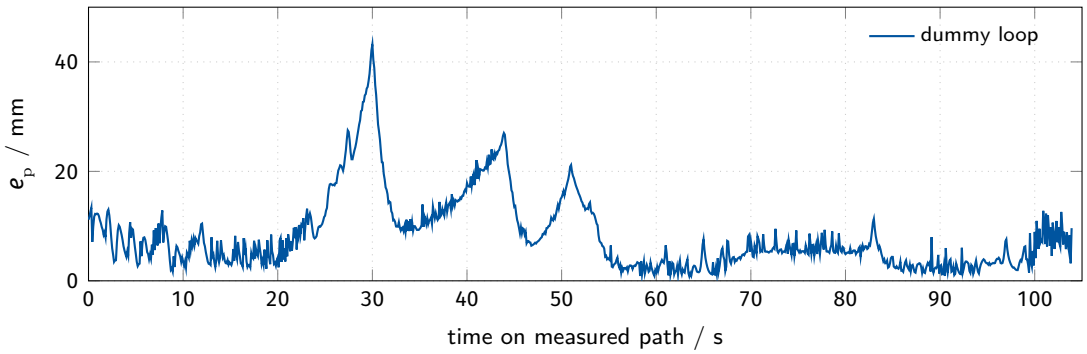


Figure 4.7: Time-aligned end-effector position error of the dummy loop over the eight measured edges, with time zero at the first sample carrying `segment_id` ≥ 1. The error stays in the millimetre range throughout. It is largest on the four edges of the lower face (0 s to 53 s) and smallest on the upper face (53 s to 104 s).

5

Hardware Deployment and Preliminary Experimental Results

The dummy loop of Chapter 4 accepts the Experiment A files on a kinematic plant. This chapter takes the same files through the MRT bridge of Section 3.4: first with mock `ros2_control` interfaces that echo commands into state, then on the physical HERo platform. The mock still has policy lag, velocity clamps and the Zenoh boundary; what it does not have is actuator dynamics or wheel-odometry drift. The gap between the two sessions is therefore the contribution of the plant and of the state estimate, on top of a residual that the dummy never sees.

Section 5.1 records how the deployment was started and which sessions are reported. Section 5.2 treats the timing and model problems that show up once the bridge is in the loop. Section 5.3 compares one mock lap with one hardware lap on the eight-corner path of Figure 4.4, using the metrics of Table 4.1. Both traces are internal metrology (Section 4.1.1). A single slow lap is not an ISO 9283 test [ISO 9283].

5.1 Deployment on the HERo Platform

The topology is that of Section 3.2: the external PC solves the optimal-control problem and publishes the Experiment A reference; the on-board IPC evaluates the policy and commands the drives. Both machines run the project containers on ROS 2 Jazzy with the Zenoh middleware. The external PC was attached to the platform subnet by Ethernet for every session below. The wireless access point of the platform remains a possible join path, and was the reason Zenoh was chosen in the first place, but it was not used as the control link here: a cable keeps the hop that gave trouble during bring-up (Section 3.2) out of the loop, at the price of tethering a 400 kg base to a desk. The box of Figure 4.4 is small enough that the cable did not go taut in the recorded laps; it is still a run condition, not a property of the controller.

Start-up follows the order already required by the reset handshake. Each machine starts its Zenoh router, the external PC router pointing at the IPC. The Experiment A host launch then brings up the MPC node, the waypoint target node and RViz, with the same task file and waypoint file the dummy used. Only after the reset service is advertised is the IPC launch started, so that the bridge can handshake immediately rather than wait. On the first start after a model change the external PC still spends about a minute compiling CppAD libraries; later starts reuse the cache. RViz is checked before anyone calls `/start`: the landed box, the orange lead legs, and in particular the lead-out at $z = 1.15\,\mathrm{m}$, have to sit in free space. The approaching gate of Section 4.2.2 then holds the reference at the lead-in until the measured end-effector is within 0.02 m of that point. Orientation is not part of the gate, so a residual attitude at entry is admitted into the first measured edge.

A mock session is the same pair of launches with `mock_arm:=true` and `mock_platform:=true` on the IPC side. The controller manager, the MRT bridge and the three OCS2 streams across Zenoh are those of the hardware stack; only the `ros2_control` hardware interfaces are replaced by echo components [ros2_control]. That is a stricter check than the dummy, and a weaker one than the robot. It is the last run that can still be repeated at a desk.

Closed-loop motion on the physical platform was obtained before the sessions reported here. Those earlier laps are omitted, because the approaching gate, the segment identifiers and the metric topic were not yet the pipeline of Chapter 4. The traces below are one mock lap and one hardware lap recorded on 14 August 2026, at the operating point of Section 4.2.2: path speed 0.05 m/s, corner dwell 2.0 s, horizon $T = 1.3\,\mathrm{s}$, solution time window $T_{\mathrm{sol}} = 1.2\,\mathrm{s}$, feed-forward SLQ with one iteration, $\mu_{\mathrm{p}} = 16$, $\mu_{\mathrm{o}} = 3$, and R_{base} pre-scale 5.5. The solver on the external PC used eight threads. Host and IPC logs were kept alongside the bags (`bag_host_0814_1558` mock, `bag_host_0814_1613` hardware). The task and waypoint files these sessions loaded are not the ones in the working tree: they are the snapshot taken from commit `a38bc0d` and archived with the recordings under `paper_data/experiment_a/20260814_005ms_8pt/config/`. The live configuration has since been retuned, so a run started from the current repository reproduces neither the operating point listed above nor the numbers below. Metrics are computed from the exported `ee_metrics` and solver-diagnostics streams, restricted to `segment_id` ≥ 1 as in Section 4.1.2.

5.2 Deployment Challenges and Mitigations

The real-time invariant of Equation (3.5) was written for the everyday file `task_hardware.info`, where $T_{\mathrm{sol}} = 1.0\,\mathrm{s}$. The Experiment A file widens that window to 1.2 s so that a slow solve can still hand the bridge a policy before the previous one expires. On the hardware lap the last-solve time had mean 227 ms, median 218 ms and 95th percentile 297 ms; the slowest sample in that stream was

353 ms, and the solver's own running maximum reached 720 ms. Policies arrived at 4.4 Hz. Twice the mean solve, 0.45 s, is still less than T_{sol}, and the bridge did not enter a policy-exhaustion hold on this lap. The loop therefore ran in real time in the sense of Equation (3.5), although policies arrived at less than half the 10 Hz named by `mpcDesiredFrequency`; on the robot that entry throttles nothing (Section 3.4).

The mock lap was faster on average (mean 174 ms, effective 5.8 Hz) but not safer in the tail. One solve took 1.12 s, which is inside the 1.2 s window and would have been past the 1.0 s window of the everyday file. The bridge logged three "policy nearly exhausted" warnings on that lap; the hardware lap logged none. OCS2 still printed its "solution time window might be shorter than the MPC delay" warning once on the robot and nine times on the mock. The warning compares the window with an internal delay estimate and is not the same test as a hold. Widening T_{sol} is what kept the spike from freezing the mock plant. It does not make a 10 Hz solver out of a 4 Hz one. Figure 5.1 puts the two distributions side by side.

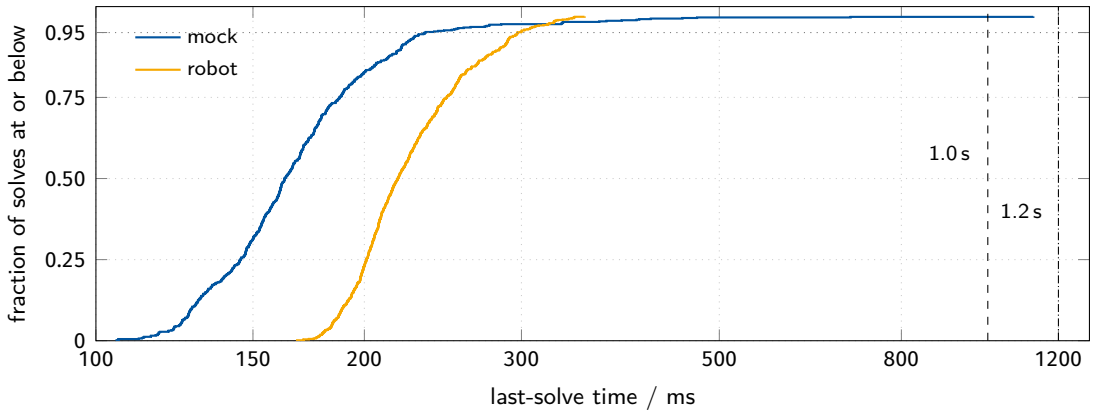


Figure 5.1: Empirical distribution of the last-solve time on the two laps, on a logarithmic axis; Figure 5.6 shows the same two streams against wall time. The mock curve sits to the left of the robot curve over most of its range—the mock solves faster at the median, 163 ms against 218 ms—but it is the mock that carries the tail: its slowest solve is 1.12 s, against 353 ms on the robot, and about one mock solve in a hundred exceeds 400 ms where the robot stream ends. The dashed line is the solution time window of `task_hardware.info` and the dash-dotted line the wider window of the Experiment A file; the mock spike falls between them, which is why it expired a policy under the everyday setting but not under the one actually used (Equation (3.5)).

Because a new policy is already old when it arrives, the input that the bridge applies is never the one at $\tau = 0$ of the horizon. Figure 5.2 sketches the timing. On the hardware lap the lag at policy arrival—solve time plus the Zenoh hop—had mean 0.24 s and 95th percentile 0.32 s after the first few samples, against a maximum of 0.36 s. Between arrivals the bridge keeps evaluating the same policy, so the lag at the interpolation site grows until the next message; that quantity had mean 0.36 s. The

mock figures were 0.18 s at arrival and 0.29 s at interpolation. In both cases the first one to two tenths of a second of every plan are unused. Diagnostics [policy_arrival] and [policy_interp] were left enabled at 0.1 s on the IPC launch so that these numbers could be read from the log rather than inferred from solve time alone.

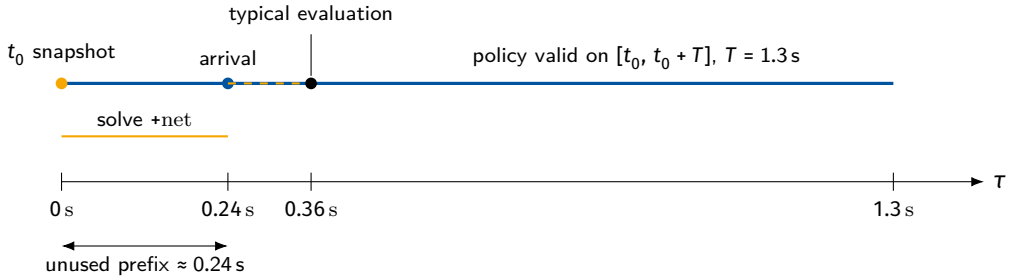


Figure 5.2: Policy lag on the hardware lap. A plan stamped at the snapshot t_0 becomes available only after the solve and the network hop, and is then read at a time that continues to advance until the next plan arrives. Times are the measured means of that session, not configuration values.

If that prefix contains the only aggressive correction the solver is willing to apply, the robot never receives it. The OCS2 model is a kinematic integrator: a large velocity at $\tau = 0$ really would cancel a position error in a few tens of milliseconds, and a high μ_p encourages exactly that pulse. For ten policies of the hardware lap the full input trajectory was recorded, not only its first sample, which makes the prefix directly readable (Figure 5.3). In all ten the base command does not merely decay across the lag: it changes sign. The forward velocity starts at a median 0.078 m/s, close to the 0.1 m/s bound, crosses zero at a median $\tau = 0.235$ s, and is a median −0.015 m/s by the 0.361 s at which the bridge interpolates. The yaw rate behaves the same way, starting at a median 0.099 rad/s, which is the soft bound itself, and reaching −0.010 rad/s. The robot is therefore not simply missing the correction the solver computed; over this sample it receives a base command of the opposite sign.

The mock lap, recorded with the same files and exported the same way, acts as the control. Its twelve policies carry a base command that starts at a median 0.008 m/s, an order of magnitude below the bound, and still reads 0.005 m/s at its own 0.29 s of interpolation lag; two of the twelve change sign, both while crossing zero on a signal that never leaves a few millimetres per second. The front-loaded, saturating correction is absent there. That is the expected consequence of the plant: when the state follows the command, the tracking error never grows enough for a high μ_p to demand a large velocity at $\tau = 0$. The two laps therefore differ not only in how late the policy is read but in what the policy contains, and the second difference is a consequence of the first acting on a plant that lags.

This sample is narrow. The ten policies span 1.8 s of one lap, and 0.361 s is the session mean rather than the lag of each individual cycle, so the figure shows a mechanism at work in a window and

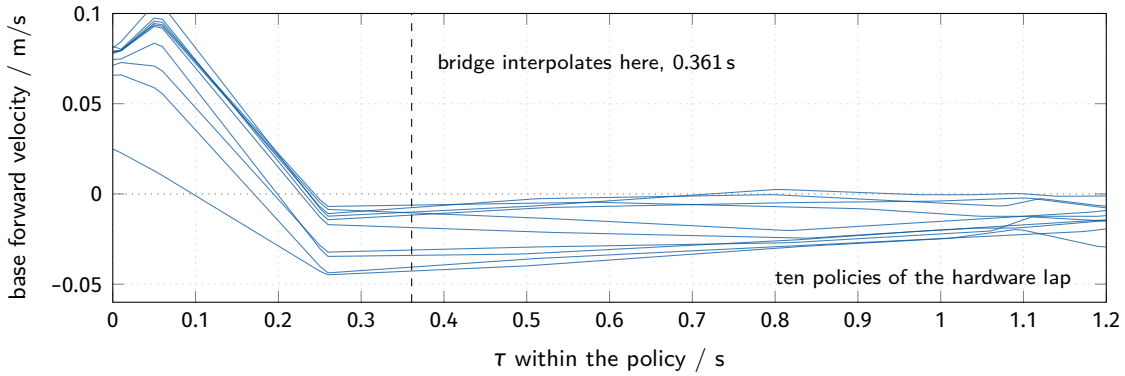


Figure 5.3: Base forward velocity along the horizon for the ten policies of the hardware lap whose full input trajectory was recorded. Every one of them opens near the 0.1 m/s bound and has already reversed sign by the dashed line, which marks the mean lag at which the bridge interpolates. The reversal is a property of the plan, not of the plant: the solver spends its correction in a prefix that the lag consumes.

not a statistic over the lap. A second, independent check covers the whole lap. Along-track lag t_{lag} of Equation (4.2) should be comparable to the interpolation lag if delay alone were the story. On the mock it is (mean t_{lag} = 0.39 s against 0.29 s of interpolation lag). On the robot it is not (mean t_{lag} = 1.81 s), which is what a command of the wrong sign, applied for part of every cycle, would produce.

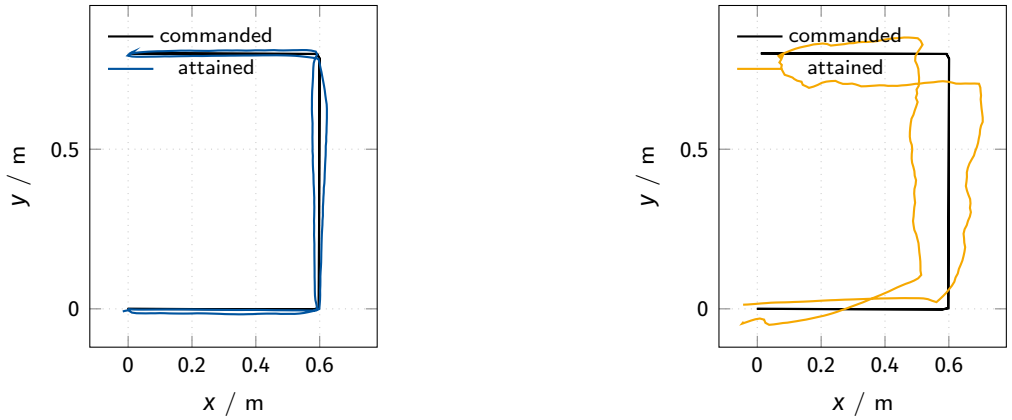
Two other failure modes of the bridge were already designed around and barely appeared in these sessions. A mid-run reset handshake still replaces the reference with a two-knot hold at the current end-effector; the waypoint node re-publishes the same knots at 1 Hz on the OCS2 time axis so that a lost reference comes back without a step in the command. The arm rate clamp of Table 3.4 issued two warnings on each lap, which is not a dominant trace. Protective stops and teach-pendant mode errors, which dominated the first bring-up of the IPC launch, are operational rather than closed-loop problems: the UR driver has to start in headless remote mode, and the controller stopper has to keep the joint-trajectory controller in its ignore list, as already required in Section 3.4. They are mentioned here only because a session that skips those checks never produces a bag.

The remaining limitation is the model. Mock hardware realises the integrator: commanded wheel and joint rates appear in the next observation. The physical base is a 400 kg platform whose drive controller enforces its own acceleration limit [HENK19], and the UR10e in streaming position control has a following error that the kinematic model does not know [UR10e]. Every re-solve on the robot therefore starts from a state the previous plan did not expect. That is a different problem from a slow solver, and Section 5.3 is mostly about it.

5.3 Preliminary End-Effector Tracking Results

Both sessions completed the eight measured edges and finished on the lead-out hold at $z = 1.15\,\mathrm{m}$. The mock approaching gate ran for 10 s from an initial end-effector error of 1.04 m; the robot needed 19 s from 0.34 m, longer than the mock from a third of the distance, and entered `RUNNING` at 19 mm with an orientation error of 0.061 rad that the gate does not wait for. From the first sample with `segment_id` ≥ 1 to the last, both laps lasted 104 s, which is the commanded 4.50 m/0.05 m/s plus seven corner dwells of 2 s (the last corner has no dwell before the lead-out). Task-completion time T of Table 4.1 therefore does not distinguish the two plants. Path shape does.

Figure 5.4 shows the commanded and attained end-effector paths in a frame attached to the first corner, with x along P1–P2 and y along the leftward horizontal edge. The vertical edges of the box collapse in this view; they remain in the error signals. On the mock the attained path follows the rectangle with a lag that is visible mainly as a short-cut at the corners. On the robot the attained path is a rounded, inward-offset copy of the same rectangle, with a consistent bias of order 10 cm rather than a delay that stays on the line.



(a) Mock hardware **(b)** Physical robot

Figure 5.4: Commanded and attained end-effector paths on the measured edges, in a frame with origin at P1 and x along P1–P2. The two vertical edges of Figure 4.4 project onto the short sides.

Table 5.1 collects the scalar scores. Time-aligned position error e_{p} on the mock had RMS 27 mm and maximum 71 mm; on the robot the same quantities were 130 mm and 204 mm. Against the 11 mm that the dummy loop leaves on the same commanded path (Table 4.3), the step to mock hardware—the bridge, its clamps, and a loop that now runs against the wall clock—is a factor of about two and a half, and the physical plant adds a further factor of five. Mean Hausdorff $\overline{H}$ rose from 12 mm to 62 mm, and the 95th percentile of the nearest-neighbour distances from 23 mm to 110 mm. The classical H is dominated by a single mock sample at 71 mm and is the weaker summary, which is why Section 4.1.3 asked for $\overline{H}$ and P_{95} as well. Orientation error grew with the plant too:

RMS 0.056 rad on the mock, 0.27 rad on the robot, with a hardware maximum of 0.47 rad on later edges. With $n = 1$ cycle, ISO path accuracy AT_p is not reported as a standard value; $h(B, A)$ on the densely resampled command is the nearest stand-in and sits at 71 mm (mock) and 123 mm (robot).

Table 5.1: Mock versus hardware scores on the measured Experiment A lap (`segment_id` ≥ 1). Hausdorff quantities use a 1 mm resampling of the commanded polyline, as in Section 4.1.3. Solve time is the last-solve sample of the diagnostics stream; policy lag at arrival is the MRT `policyLagAtArrival` log, excluding the first five samples.

	Mock hardware	Physical robot
e_{p} RMS (max)	27 mm (71 mm)	130 mm (204 mm)
e_{o} RMS (max)	0.056 rad (0.13 rad)	0.27 rad (0.47 rad)
H / $\overline{H}$ / P_{95}	71 mm / 12 mm / 23 mm	135 mm / 62 mm / 110 mm
mean t_{lag}	0.39 s	1.81 s
mean solve time (effective rate)	174 ms (5.8 Hz)	227 ms (4.4 Hz)
mean policy lag at arrival	0.18 s	0.24 s
measured lap T	104 s	104 s

Figure 5.5 plots $e_{\mathrm{p}}(t)$ on the measured edges, with time zero at the first admitted sample. The mock trace stays below 50 mm except for a short spike at the first corner. The hardware trace climbs through the first long edge, plateaus around 15 cm to 20 cm on edges 2–4 and 6, and only comes back toward 5 cm on the last horizontal edge. It is not a delayed copy of the mock.

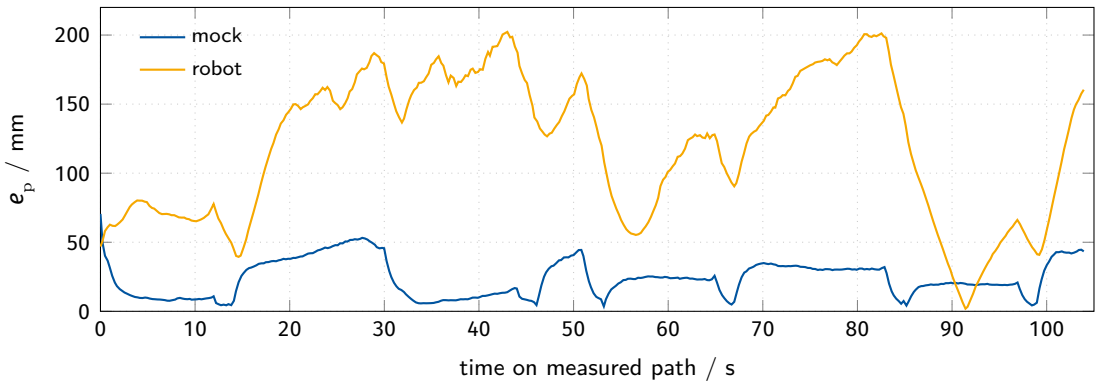


Figure 5.5: Time-aligned end-effector position error on the measured path. Time zero is the first sample with `segment_id` ≥ 1.

Per-edge numbers make the same split (Table 5.2). The mock RMS stays between 10 mm and 41 mm on every edge. The robot is already at 69 mm RMS on the opening 0.60 m edge and peaks at 175 mm on edge 3. The two vertical edges (4 and 8), which the base cannot help with, are not the worst

hardware edges; the long horizontal edges are. Along-track error on the robot is positive on seven of the eight edges and has mean 91 mm, which at $v_{\mathrm{ref}} = 0.05\,\mathrm{m/s}$ is the 1.81 s of t_{lag} already quoted. A pure policy-lag prediction $0.36\,\mathrm{s} \times v_{\mathrm{ref}}$ is only 18 mm. The mock along-track mean, 19 mm, matches that prediction. The extra 70 mm on the robot is not delay in the MRT bridge.

Table 5.2: Per-edge end-effector position error on the measured lap. Edges 1, 3, 5 and 7 are the 0.60 m sides; 2 and 6 are the 0.80 m sides; 4 and 8 are the vertical 0.25 m sides.

Edge	Mock RMS	Mock max	Robot RMS	Robot max
1	16 mm	71 mm	69 mm	80 mm
2	41 mm	53 mm	143 mm	187 mm
3	10 mm	18 mm	175 mm	204 mm
4	31 mm	45 mm	145 mm	174 mm
5	22 mm	26 mm	98 mm	129 mm
6	30 mm	35 mm	169 mm	202 mm
7	18 mm	21 mm	60 mm	140 mm
8	39 mm	45 mm	111 mm	160 mm

Corner dwells isolate the same point without mixing it into a moving reference. On the mock, the last 2 s of edges 1–7—the dwell windows—fall to between 5 mm and 25 mm. On the robot the same windows remain between 53 mm and 168 mm and do not collapse. Once the command has stopped, a lag of 0.36 s should have died out. What remains is a steady offset that the receding-horizon loop does not remove at this operating point.

Solve time over the two sessions is shown in Figure 5.6 only to close the timing discussion. After the first seconds both traces sit near their means; the mock spike to 1.12 s is the event that produced the exhaustion warnings. Neither curve tracks $e_{\mathrm{p}}(t)$. A slow solver would have frozen the robot through the hold logic of Table 3.4. That did not happen. The tracking gap is left by the plant and by the lag acting on a model that assumes the plant is an integrator.

Two caveats bound what these numbers may be used for. First, both desired and attained poses are reconstructed from the MPC observation, so wheel-odometry drift is shared by the command frame and the measured end-effector. On the robot the box was landed at a base pose near (23.2, 9.0) in the odometry frame, which is the accumulated motion of the platform before this lap, not a localisation in the hall. Closed-loop drift of the attained path relative to its own start was 15 cm on the robot and 11 cm on the mock, most of it in z at the last vertical edge rather than a planar loop-closure error. None of this is an absolute accuracy statement in the hall. Second, each plant

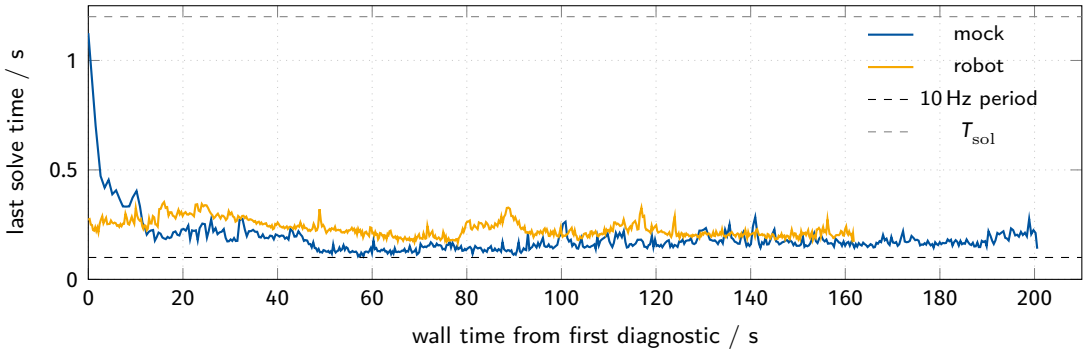


Figure 5.6: Last-solve time on the mock and hardware sessions. The lower dashed line is the 0.1 s period of `mpcDesiredFrequency`, which the bridge uses only as its warning threshold; the upper line is $T_{\mathrm{sol}} = 1.2\,\mathrm{s}$.

was run once, at one speed. Repeatability RT_p and the three speed levels of ISO Table 3 are outside the present data [ISO 9283].

What the pair of laps does show is narrower. The Experiment A files that the dummy accepted can be executed by the hardware stack from approaching gate to lead-out hold. Mock hardware, which shares the bridge and the lag with the robot, tracks at a few centimetres and at an along-track delay consistent with policy lag. The physical platform, with the same files, tracks at one to two decimetres and does not catch the reference during dwell. The next improvements that would change those numbers are a shorter solve (so that $u(\tau = 0)$ is no longer in the past) and a model that knows the base cannot step in velocity.

6

Conclusion and Outlook

This chapter closes the thesis by returning to the objective of Section 1.3: integrate whole-body MPC on the physical HERo platform and characterise its end-effector-tracking behaviour. The preceding chapters already separate what the optimiser does on its kinematic model (Chapter 4), what the MRT bridge adds (Section 5.2), and what the plant adds on top (Section 5.3). What follows is a concise account of what was achieved, what the present data do and do not establish, and which engineering steps would most directly change the numbers reported in Chapter 5.

6.1 Summary

The work set out to move OCS2 whole-body MPC from a framework demonstration to a deployable stack on a differential-drive mobile manipulator in an LMAS-oriented setting. Against that goal, four contributions were delivered.

First, a whole-body motion-planning approach was selected for the HERo platform after comparing OCS2 with alternative frameworks (Chapter 2). Second, OCS2 was integrated into the existing software stack: a deployment over Zenoh keeps the actuation path local to the IPC while the SLQ solver runs on an external PC (Section 3.2); a nine-state kinematic model with non-holonomic base, safety-panel joint limits, and a reduced self-collision set was parameterised for on-robot operation (Section 3.3); and an MRT bridge turns the published policy into wheel and joint commands under explicit guards (Section 3.4). Third, a simulation-based acceptance path was built around Experiment A: the task file and waypoint list that the robot loads are exercised in a dummy MRT loop, internal metrology and ISO-oriented path metrics score that loop and the hardware laps alike, and on the archived files the dummy loop tracks the commanded box at 11 mm RMS (Table 4.3). Fourth, that configuration was taken through mock `ros2_control` hardware and onto the physical platform. The closed eight-corner path of Figure 4.4 completed on both plants; the approaching gate, segment tagging, and lead-out hold that Section 4.2.2 specifies all appeared in the recorded sessions.

The result these contributions produce is a gap, and the gap is the outcome this work reports. The same Experiment A files were executed on three plants that differ only in what lies below the policy. The dummy loop, where nothing but the optimiser acts, tracks the commanded box at 11 mm RMS (Table 4.3). Mock hardware, which adds the bridge, its clamps and a loop bound to the wall clock, tracks at 27 mm. The physical platform, with the same files and the same lap duration, tracks at 130 mm and does not recover during the corner dwells (Figure 5.5 and Tables 5.1 and 5.2). Geometry, the OCS2 time axis and the cost tuning are common to all three and are therefore not what separates them.

What separates them is where the controller's correction sits inside the plan. The loop ran in real time in the sense of Equation (3.5), with no policy-exhaustion hold on the hardware lap, but at an effective 4.4 Hz set by the solve time, so the bridge reads each policy a mean 0.361 s after the state it was computed for (Figure 5.2). For the ten policies of that lap whose full input trajectory was recorded, the base command does not merely decay over that interval: it opens near the 0.1 m/s bound, crosses zero at a median $\tau = 0.235$ s, and has reversed sign by the time the bridge samples it (Figure 5.3). Over that sample the robot receives a base command opposite to the one the solver computed. The mock lap, read the same way, carries no such pulse at all: its base command starts an order of magnitude below the bound and barely changes across its own lag. Both plants read their policies late. Only the physical platform provokes a plan that is unsafe to read late.

Several constraints bound how strongly these results can be read. All desired and attained poses were reconstructed from wheel odometry and joint encoders inside the waypoint node (Section 4.1.1), so the scores measure internal consistency of the closed loop, not absolute accuracy in the hall. Each plant was exercised once, at $v_{\mathrm{ref}} = 0.05$ m/s, without the three mandatory ISO speed levels or repeated cycles [ISO 9283]. The PC was connected by Ethernet throughout (Section 5.1); the numbers therefore do not speak to control over the platform's wireless link. The OCS2 model treats the base as a velocity integrator and knows nothing of the drive controller's acceleration limit or of the UR10e following error under streaming position control [HENK19; UR10e].

Figure 6.1 collects the requirements R1 to R7 of Table 1.1, what realises each one in the integrated system, and the evidence reported above for it.

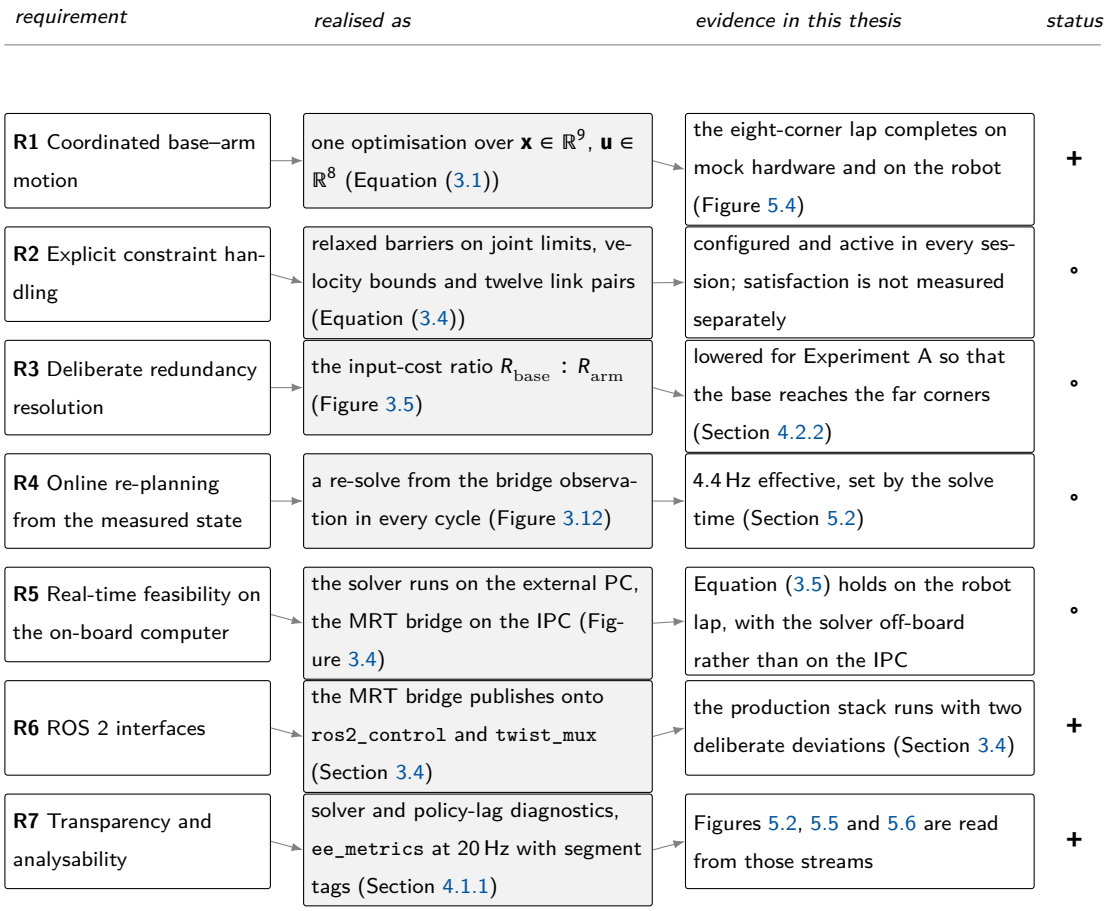


Figure 6.1: The requirements of Table 1.1, what realises each one in the integrated system, and the evidence this thesis offers for it. The symbols are those of Table 2.1: + met, ∘ met in part. R2 and R3 are implemented and in use, but no measurement scores them on their own. R4 is met at the rate the solve time allows, not at a rate the task file sets. R5 is the requirement whose wording the deployment does not meet: the loop satisfies Equation (3.5), yet it does so because the solver was moved off the on-board computer, so the requirement holds for the robot as a whole and not for the IPC.

6.2 Critical Reflection

The experiments answer a narrower question than the LMAS motivation of Section 1.1 might suggest. They show that a time-parameterised end-effector reference, published as OCS2 `TargetTrajectories` rather than as a sequence of static goals, can be executed end to end on HERo hardware by a receding-horizon whole-body controller, and they localise the largest tracking gap to the plant and to the policy lag acting on a model that treats that plant as an integrator, rather than to a broken data pipeline or to a reference that the optimiser cannot track on its own model. They do not, by themselves, establish that this pipeline meets production accuracy requirements, nor

that whole-body MPC outperforms a well-tuned decoupled stack on task completion time or corner accuracy.

The metric layer deserves the same caution. Time-aligned pose error e_{p} and the along-track lag t_{lag} of Equation (4.2) are meaningful comparisons *within* one controller, because desired and attained poses share the same observation time and the same state estimate. Hausdorff distance and its mean and percentile variants (Equation (4.4)) remove the assumption of a common speed profile, but with $n = 1$ cycle they still do not yield ISO path accuracy AT_p or repeatability RT_p in the sense of ISO 9283 [ISO 9283].

The split between mock and hardware results is instructive in what it rules out. Mock hardware preserves policy lag, velocity clamps, and the Zenoh hop; since it tracks at 27 mm RMS, the reference geometry, the OCS2 time axis, and the cost tuning are not the dominant fault. What remains has two parts, and they carry different weight. The sign reversal of Figure 5.3 is measured, and it explains why an error that exceeds the pure lag prediction $0.36\,\mathrm{s} \times v_{\mathrm{ref}}$ can persist through a dwell: a command of the wrong sign does not stop being wrong when the reference stops moving. Why the solver places its correction there is the second part, and it is still inferred rather than shown. The kinematic integrator of Section 3.3 lets the base velocity step to its 0.1 m/s bound at $\tau = 0$, and nothing in the model charges the 400 kg platform for the acceleration that step would require, so a plan that front-loads the correction is optimal on the model and infeasible on the robot. Separating that from wheel slip, odometry drift and arm tracking error would need additional sensing or deliberately simpler motions.

Scope limits are finally worth stating plainly. One mobile manipulator, one closed path, and one lap per plant are enough to debug integration and to expose the main failure modes; they are not enough to support statistical claims about repeatability or about behaviour across the workspace. The waypoint node of Section 4.2.2 extends the upstream OCS2 example, which regulates to a single goal set by an interactive marker in RViz, to the tracking of a time-parameterised trajectory [OCS2]. The evaluation has so far exercised only that tracking regime, on a single box geometry.

6.3 Recommendations for Resolving Present Issues

The hardware lap already identifies where effort would change the recorded numbers, rather than merely widen the solution time window.

Raise the effective MPC rate and shorten policy lag.

Mean solve time on the hardware session was 227 ms (Table 5.1), so the controller replans at roughly 4.4 Hz, a rate set by the solve time rather than by the task file (Section 3.4). The DDP benchmark

that OCS2 prints at shutdown attributes that time to two stages above all. In an earlier instrumented session on the same node, summarised over 1794 iterations, *Initialization* averaged 170 ms per iteration (56 % of solver runtime) and *Search Strategy* 111 ms (36 %), while the backward pass stayed below 1 ms. The absolute values belong to that session rather than to the laps of Chapter 5, but the split suggests attacking warm-start cost and rollout search before adding SLQ iterations. Pinning the solver threads away from unrelated PC load, keeping the Release build used here, and revisiting the twelve-pair self-collision set if profiling shows that distance queries dominate those two stages are practical first steps. Enabling a feedback policy (`useFeedbackPolicy=true` in the task file; currently disabled in Table 3.3) would let the bridge use linear gains between sparse policy updates [GRAN19; OCS2]; the trade-off is larger policy messages over Zenoh, which must be measured on the link that will actually be used in production. Widening T_{sol} further, as in the Experiment A file, prevents exhaustion when a solve spikes; it does not restore the aggressive $\tau = 0$ input that lag already consumes (Figure 5.2).

Close the model–plant gap on the base.

The two largest per-edge residuals on the robot are on horizontal edges (edges 3 and 6 of Table 5.2), where the base can take part in the motion, and the residual does not relax when the reference stops. Adding an acceleration limit or a first-order lag on base velocities inside the OCS2 model would align the optimiser with what the freeMoVe drive stack enforces and reduce the state mismatch that each re-solve inherits [HENK19]. A complementary experiment—base motion on a closed planar path with the end-effector held fixed in space, the converse of Experiment A—would separate base tracking from arm-dominated edges without changing the software architecture.

Improve pose estimation.

If odometry drift dominates closed-loop drift, fusing wheel odometry with additional proprioceptive or exteroceptive sources through an extended Kalman filter (for example the `robot_localization` package) is a direct next step; the present pipeline uses raw wheel odometry only (Section 3.1) [MOOR15].

Complete the evaluation programme.

Two items would complete the present characterisation. ISO Table 3 speed levels and $n \geq 3$ cycles on the whole-body pipeline would populate AT_p and RT_p [ISO 9283]. Re-running the mock versus hardware comparison over a wireless Zenoh link would test whether the Ethernet sessions of Section 5.1 bias the timing budget optimistically [Zenoh].

Run the lag-compensation experiment first.

One experiment is prior to every item above. The sign reversal of Figure 5.3 predicts that the hardware residual is driven by *when* the policy is read rather than by what the plant does with it, and that prediction is falsifiable on the same robot with the same files: re-run the lap with the lag compensated, and measure whether the 130 mm moves towards the 27 mm of the mock plant. The lag can be compensated by the feedback policy named above, by starting each solve from the state predicted for the instant at which its policy will be read, or by a solve short enough that the bridge reads the prefix it now skips. If the residual moves, the gap is a timing problem and the model can stay kinematic. If it does not, the correction sits in the model, and an acceleration-limited base is the next thing to build. The recorded laps cannot decide between these two readings (Section 6.2); this measurement can. Figure 6.2 lays out that decision.

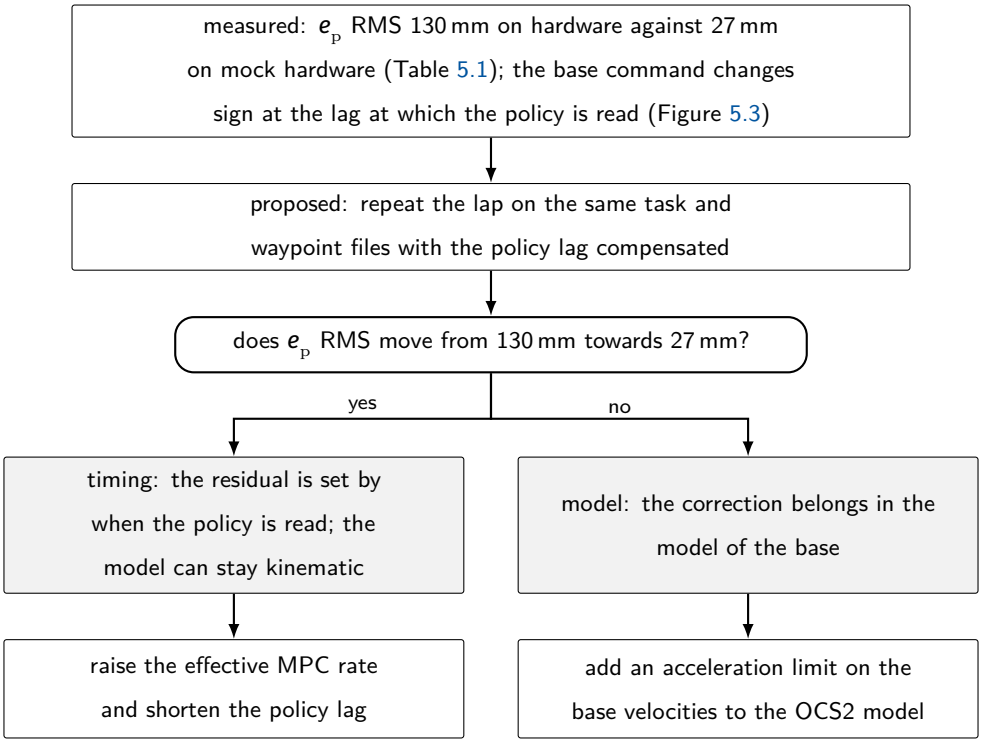


Figure 6.2: Decision settled by the lag-compensation experiment. The two values are those of Table 5.1. The experiment itself has not been run; the lower two rows are the outcomes it would separate and the recommendation of this section that each outcome leads to.

References

[BUCK19] A. F. Buckhorst, G. Hüttemann, L. Grahn, and R. H. Schmitt. "Assignment, Sequencing and Location Planning in Line-less Mobile Assembly Systems". In: *Tagungsband des 4. Kongresses Montage Handhabung Industrieroboter*. Springer Berlin Heidelberg, 2019, pp. 227–238. DOI: 10.1007/978-3-662-59317-2_23.

[BUCK22] A. F. Buckhorst, L. Grahn, and R. H. Schmitt. "Decentralized Holonic Control System Model for Line-less Mobile Assembly Systems". In: *Robotics and Computer-Integrated Manufacturing* 75 (2022), p. 102301. DOI: 10.1016/j.rcim.2021.102301.

[BURG24] B. Burgess-Limerick, J. Haviland, C. Lehnert, and P. Corke. "Reactive Base Control for On-the-Move Mobile Manipulation in Dynamic Environments". In: *IEEE Robotics and Automation Letters* 9.3 (2024), pp. 2048–2055. DOI: 10.1109/LRA.2024.3354615.

[CARP19] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiraux, O. Stasse, and N. Mansard. "The Pinocchio C++ Library: A Fast and Flexible Implementation of Rigid Body Dynamics Algorithms and Their Analytical Derivatives". In: *2019 IEEE/SICE International Symposium on System Integration (SII)*. IEEE, 2019, pp. 614–619. DOI: 10.1109/SII.2019.8700380.

[chrony] chrony development team. *chrony – Versatile Implementation of the Network Time Protocol*. https://chrony-project.org/. Accessed: Tuesday 13th October, 2026. 2024.

[CppAD] B. M. Bell. *CppAD: A Package for Differentiation of C++ Algorithms*. https://coin-or.github.io/CppAD/. COIN-OR project. Accessed: Tuesday 13th October, 2026. 2024.

[DIET15] A. Dietrich, C. Ott, and A. Albu-Schäffer. "An Overview of Null Space Projections for Redundant, Torque-Controlled Robots". In: *The International Journal of Robotics Research* 34.11 (2015), pp. 1385–1400. DOI: 10.1177/0278364914566516.

[DUBU94] M.-P. Dubuisson and A. K. Jain. "A Modified Hausdorff Distance for Object Matching". In: *Proceedings of 12th International Conference on Pattern Recognition*. Vol. 1. Crossref records no issued date-parts; year taken from the 12th ICPR proceedings (1994). IEEE, 1994, pp. 566–568. DOI: 10.1109/ICPR.1994.576361.

[ELMA05] H. A. ElMaraghy. "Flexible and Reconfigurable Manufacturing Systems Paradigms". In: *International Journal of Flexible Manufacturing Systems* 17.4 (2005), pp. 261–276. DOI: 10.1007/s10696-006-9028-7.

[ELMA13] H. ElMaraghy, G. Schuh, W. ElMaraghy, F. Piller, P. Schönsleben, M. Tseng, and A. Bernard. "Product Variety Management". In: *CIRP Annals* 62.2 (2013), pp. 629–652. DOI: 10.1016/j.cirp.2013.05.007.

[FARS17] F. Farshidian, M. Neunert, A. W. Winkler, G. Rey, and J. Buchli. "An Efficient Optimal Planning and Control Framework for Quadrupedal Locomotion". In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 93–100. DOI: 10.1109/ICRA.2017.7989016.

[FELL17] C. Feller and C. Ebenbauer. "Relaxed Logarithmic Barrier Function Based Model Predictive Control of Linear Systems". In: *IEEE Transactions on Automatic Control* 62.3 (2017), pp. 1223–1238. DOI: 10.1109/TAC.2016.2582040.

[FINE23] M. N. Finean, L. Petrović, W. Merkt, I. Marković, and I. Havoutis. "Motion Planning in Dynamic Environments Using Context-Aware Human Trajectory Prediction". In: *Robotics and Autonomous Systems* 166 (2023), p. 104450. DOI: 10.1016/j.robot.2023.104450.

[GIFT17] M. Giftthaler, F. Farshidian, T. Sandy, L. Stadelmann, and J. Buchli. "Efficient Kinematic Planning for Mobile Manipulators with Non-Holonomic Constraints Using Optimal Control". In: *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 3411–3417. DOI: 10.1109/ICRA.2017.7989388.

[GRAN19] R. Grandia, F. Farshidian, R. Ranftl, and M. Hutter. "Feedback MPC for Torque-Controlled Legged Robots". In: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 4730–4737. DOI: 10.1109/IROS40897.2019.8968251.

[HAVI21] J. Haviland and P. Corke. "NEO: A Novel Expeditious Optimisation Algorithm for Reactive Motion Control of Manipulators". In: *IEEE Robotics and Automation Letters* 6.2 (2021), pp. 1043–1050. DOI: 10.1109/LRA.2021.3056060.

[HAVI22] J. Haviland, N. Sünderhauf, and P. Corke. "A Holistic Approach to Reactive Mobile Manipulation". In: *IEEE Robotics and Automation Letters* 7.2 (2022), pp. 3122–3129. DOI: 10.1109/LRA.2022.3146554.

[HENK19] HENKEL + ROTH GmbH. *Betriebsanleitung HR-Projekt 569 – Mobiler Roboter "freeMoVe" (569-01 Mobiler Roboter)*. V01. Original operating manual, 04.12.2019; internal documentation of the HERo mobile base. HENKEL + ROTH GmbH. Ilmenau, Germany, 2019.

[HERT20] R. Hertwig. *Betriebsanleitung mobile robot mr 3.0*. ROS control-centre operating manual for the HERo mobile base; internal documentation, January 2020. HENKEL + ROTH GmbH. Ilmenau, Germany, 2020.

[HÜTT16] G. Hüttemann, C. Gaffry, and R. H. Schmitt. "Adaptation of Reconfigurable Manufacturing Systems for Industrial Assembly – Review of Flexibility Paradigms, Concepts, and Outlook". In: *Procedia CIRP* 52 (2016), pp. 112–117. DOI: 10.1016/j.procir.2016.07.021.

[HÜTT19] G. Hüttemann, A. F. Buckhorst, and R. H. Schmitt. "Modelling and Assessing Lineless Mobile Assembly Systems". In: *Procedia CIRP* 81 (2019), pp. 724–729. DOI: 10.1016/j.procir.2019.03.184.

[HUTT93] D. P. Huttenlocher, G. A. Klanderman, and W. J. Rucklidge. "Comparing Images Using the Hausdorff Distance". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 15.9 (1993), pp. 850–863. DOI: 10.1109/34.232073.

[ISO 9283] International Organization for Standardization. *ISO 9283:1998. Manipulating Industrial Robots – Performance Criteria and Related Test Methods*. International standard. Geneva, Switzerland, 1998.

[KAVR96] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars. "Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces". In: *IEEE Transactions on Robotics and Automation* 12.4 (1996), pp. 566–580. DOI: 10.1109/70.508439.

[KERN16] W. Kern, F. Rusitschka, and T. Bauernhansl. "Planning of Workstations in a Modular Automotive Assembly System". In: *Procedia CIRP* 57 (2016), pp. 327–332. DOI: 10.1016/j.procir.2016.11.057.

[KHAT86] O. Khatib. "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots". In: *The International Journal of Robotics Research* 5.1 (1986), pp. 90–98. DOI: 10.1177/027836498600500106.

[KORE99] Y. Koren, U. Heisel, F. Jovane, T. Moriwaki, G. Pritschow, G. Ulsoy, and H. Van Brussel. "Reconfigurable Manufacturing Systems". In: *CIRP Annals* 48.2 (1999), pp. 527–540. DOI: 10.1016/S0007-8506(07)63232-6.

[LAVA01] S. M. LaValle and J. J. Kuffner. "Randomized Kinodynamic Planning". In: *The International Journal of Robotics Research* 20.5 (2001), pp. 378–400. DOI: 10.1177/02783640122067453.

[LAVA06] S. M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006. DOI: 10.1017/CBO9780511546877.

[LYNC17] K. M. Lynch and F. C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017. DOI: 10.1017/9781316661239.

[MACE20] S. Macenski, F. Martin, R. White, and J. G. Clavero. "The Marathon 2: A Navigation System". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 2718–2725. DOI: 10.1109/IROS45743.2020.9341207.

[MACE22] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. "Robot Operating System 2: Design, Architecture, and Uses in the Wild". In: *Science Robotics* 7.66 (2022). DOI: 10.1126/scirobotics.abm6074.

[MAYN00] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. "Constrained Model Predictive Control: Stability and Optimality". In: *Automatica* 36.6 (2000), pp. 789–814. DOI: 10.1016/S0005-1098(99)00214-9.

[MINN19] M. V. Minniti, F. Farshidian, R. Grandia, and M. Hutter. "Whole-Body MPC for a Dynamically Stable Mobile Manipulator". In: *IEEE Robotics and Automation Letters* 4.4 (2019), pp. 3687–3694. DOI: 10.1109/LRA.2019.2927955.

[MINN21] M. V. Minniti, R. Grandia, K. Fäh, F. Farshidian, and M. Hutter. "Model Predictive Robot-Environment Interaction Control for Mobile Manipulation Tasks". In: *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 1651–1657. DOI: 10.1109/ICRA48506.2021.9562066.

[MITT22] M. Mittal, D. Hoeller, F. Farshidian, M. Hutter, and A. Garg. "Articulated Object Interaction in Unknown Scenes with Whole-Body Mobile Manipulation". In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 1647–1654. DOI: 10.1109/IROS47612.2022.9981779.

[MOOR15] T. Moore and D. Stouch. "A Generalized Extended Kalman Filter Implementation for the Robot Operating System". In: *Intelligent Autonomous Systems 13*. Vol. 302. Advances in Intelligent Systems and Computing. Springer, 2015, pp. 335–348. DOI: 10.1007/978-3-319-08338-4_25.

[MoveIt 2] PickNik Robotics. *MoveIt 2: Motion Planning Framework for ROS 2*. https://moveit.ai. Accessed: Tuesday 13th October, 2026. 2024.

[MoveIt Pro] PickNik Robotics. *MoveIt Pro: Commercial Robot Behaviour Development Platform*. https://picknik.ai/pro/. Proprietary, source-available under commercial licence. Accessed: Tuesday 13th October, 2026. 2025.

[OCS2] F. Farshidian et al. *OCS2: An Open Source Library for Optimal Control of Switched Systems*. https://github.com/leggedrobotics/ocs2. Robotic Systems Lab, ETH Zürich. Accessed: Tuesday 13th October, 2026. 2017.

[PANK20] J. Pankert and M. Hutter. "Perceptive Model Predictive Control for Continuous Mobile Manipulation". In: *IEEE Robotics and Automation Letters* 5.4 (2020), pp. 6177–6184. DOI: 10.1109/LRA.2020.3010721.

[RAWL17] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. 2nd ed. Madison, WI: Nob Hill Publishing, 2017.

[ROS 2] Open Robotics. *ROS 2 Documentation: Jazzy Jalisco*. https://docs.ros.org/en/jazzy/. Accessed: Tuesday 13th October, 2026. 2024.

[ros2_control] ros2_control Development Team. *ros2_control: Flexible Framework for Real-Time Control*. https://control.ros.org. Accessed: Tuesday 13th October, 2026. 2024.

[SAND22] T. Sandakalum and M. H. Ang. "Motion Planning for Mobile Manipulators—A Systematic Review". In: *Machines* 10.2 (2022), p. 97. DOI: 10.3390/machines10020097.

[SICI09] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. *Robotics: Modelling, Planning and Control*. Advanced Textbooks in Control and Signal Processing. Springer London, 2009. DOI: 10.1007/978-1-84628-642-1.

[SLEI21] J.-P. Sleiman, F. Farshidian, M. V. Minniti, and M. Hutter. "A Unified MPC Framework for Whole-Body Dynamic Locomotion and Manipulation". In: *IEEE Robotics and Automation Letters* 6.3 (2021), pp. 4688–4695. DOI: 10.1109/LRA.2021.3068908.

[STEL22] S. Stelter, G. Bartels, and M. Beetz. "An Open-Source Motion Planning Framework for Mobile Manipulators Using Constraint-Based Task Space Control with Linear MPC". In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 1671–1678. DOI: 10.1109/IROS47612.2022.9982245.

[STUE21] M. Stueben, A. Hoffmann, and W. Reif. "Constraint-Based Whole-Body-Control of Mobile Manipulators in Human-Centered Environments". In: *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2021, pp. 1–8. DOI: 10.1109/ETFA45728.2021.9613281.

[ŞUCA12] I. A. Şucan, M. Moll, and L. E. Kavraki. "The Open Motion Planning Library". In: *IEEE Robotics & Automation Magazine* 19.4 (2012), pp. 72–82. DOI: 10.1109/MRA.2012.2205651.

[THAK23] S. Thakar, S. Srinivasan, S. Al-Hussaini, P. M. Bhatt, P. Rajendran, Y. J. Yoon, N. Dhanaraj, and R. K. Malhan. "A Survey of Wheeled Mobile Manipulation: A Decision-Making Perspective". In: *Journal of Mechanisms and Robotics* 15.2 (2023). DOI: 10.1115/1.4054611.

[twist_mux] ROS community. *twist_mux: Twist Multiplexer*. https://index.ros.org/p/twist_mux/. ROS package documentation. Accessed: Tuesday 13th October, 2026. 2024.

[UR10e] Universal Robots. *UR10e Technical Specifications*. https://www.universal-robots.com/products/ur10-robot/. Accessed: Tuesday 13th October, 2026. 2024.

[wbc_ros] ARC-OPT Development Team. *wbc_ros: ROS 2 Wrappers for the ARC-OPT Whole-Body Control Library*. https://github.com/ARC-OPT/wbc_ros. Accessed: Tuesday 13th October, 2026. 2024.

[WoLF] G. Graiola and WoLF Development Team. *WoLF: Whole-Body Locomotion Framework for Quadruped Robots and Wheeled Platforms*. https://github.com/graiola/wolf-setup. Accessed: Tuesday 13th October, 2026. 2023.

[YOSH85] T. Yoshikawa. "Manipulability of Robotic Mechanisms". In: *The International Journal of Robotics Research* 4.2 (1985), pp. 3–9. DOI: 10.1177/027836498500400201.

[Zenoh] ZettaScale Technology. *Zenoh: Zero Overhead Network Protocol*. https://zenoh.io. Accessed: Tuesday 13th October, 2026. 2024.